

POLITECNICO
MILANO 1863

RASD by Hamidreza Khodadadi and
Mohammad Rashidkhan

# Requirement Analysis and Specification Document

| | |
|---|---|
| **Deliverable:** | RASD |
| **Title:** | Requirement Analysis and Verification Document |
| **Authors:** | Mohammad Rashid Khan Mofrad and Hamidreza Khodadadi |
| **Version:** | 1.0 |
| **Date:** | 23-December-2025 |
| **Download page:** | https://github.com/hamidkhd/KhodadadiRashidkhan/blob/main/README.md |
| **Copyright:** | Copyright © 2025, Mohammad Rashid Khan Mofrad and Hamidreza Khodadadi – All rights reserved |

# Contents

**Table of Contents** . . . 3

**List of Figures** . . . 5

**List of Tables** . . . 5

**1 Introduction** . . . 6
- 1.1 Purpose . . . 6
- 1.2 Scope . . . 7
  - 1.2.1 World Phenomena . . . 7
  - 1.2.2 Shared Phenomena . . . 7
- 1.3 Definitions, Acronyms, Abbreviations . . . 8
  - 1.3.1 Definitions . . . 8
  - 1.3.2 Acronyms . . . 10
  - 1.3.3 Abbreviations . . . 10
- 1.4 Revision History . . . 10
- 1.5 Reference Documents . . . 10
- 1.6 Document Structure . . . 11
- 1.7 Use of Generative AI Tools . . . 11
  - 1.7.1 Tool(s) Used . . . 11
  - 1.7.2 Inputs Provided . . . 11
  - 1.7.3 Outputs Obtained . . . 11
  - 1.7.4 Verification, Refinement, and Integration . . . 12

**2 Overall Description** . . . 13
- 2.1 Product Perspective . . . 13
  - 2.1.1 Scenarios . . . 13
  - 2.1.2 Domain-level Class Diagram . . . 15
  - 2.1.3 State Diagrams . . . 16
- 2.2 Product Functions . . . 20
  - 2.2.1 Trip Management . . . 20
  - 2.2.2 Manual Path Information Management . . . 21
  - 2.2.3 Automated Path Data Acquisition and Confirmation . . . 21
  - 2.2.4 Path Search, Visualization and Recommendation . . . 21
  - 2.2.5 User and Access Management . . . 22
- 2.3 User Characteristics . . . 22
  - 2.3.1 Registered Cyclist . . . 22
  - 2.3.2 Non-Registered User . . . 23
  - 2.3.3 System Administrator . . . 23
- 2.4 Domain Assumptions, Dependencies, and Constraints . . . 23
  - 2.4.1 Domain Assumptions . . . 23
  - 2.4.2 Dependencies . . . 24
  - 2.4.3 Constraints . . . 24

**3 Specific Requirements** . . . 26
- 3.1 External Interface Requirements . . . 26
  - 3.1.1 User Interfaces . . . 26
  - 3.1.2 Hardware Interfaces . . . 27
  - 3.1.3 Software Interfaces . . . 27

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

3.1.4 Communication Interfaces . . . 28
3.2 Functional Requirements . . . 28
3.2.1 List of Functional Requirements . . . 29
3.2.2 Use Case Diagram . . . 32
3.2.3 Use Cases . . . 33
3.2.4 Sequence Diagrams . . . 45
3.2.5 Requirements Mapping . . . 57
3.3 Performance Requirements . . . 58
3.4 Design Constraints . . . 59
3.4.1 Standards compliance . . . 59
3.4.2 Hardware limitations . . . 59
3.4.3 Other constraints . . . 59
3.5 Software System Attributes . . . 59
3.5.1 Reliability . . . 59
3.5.2 Availability . . . 60
3.5.3 Security . . . 60
3.5.4 Maintainability . . . 60
3.5.5 Portability . . . 60

**4 Formal Analysis Using Alloy . . . 61**
4.1 Results . . . 66

**5 Effort Spent . . . 68**

**6 References . . . 69**

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

# List of Figures

1 Domain class diagram . . . 16
2 Trip State Diagram . . . 17
3 Detected Issue state Diagram . . . 18
4 Path State Diagram . . . 18
5 Obstacle State Diagram Obstacle lifecycle from creation to publication. . . . 19
6 Route query lifecycle from composition to answer. . . . 20
7 BBP Use Case Diagram . . . 33
8 Sequence diagram for “Register as a User” . . . 46
9 Sequence diagram for “Log in as a User” . . . 47
10 Sequence diagram for “Record a Trip” . . . 48
11 Sequence diagram for “Define a Path Manually” . . . 49
12 Sequence diagram for “Review Detected Issues” . . . 50
13 Sequence diagram for “Search Bike Paths Between Origin and Destination” . . . 51
14 Sequence diagram for “Manage Profile and Privacy Settings” . . . 52
15 Sequence diagram for “View Trip History and Statistics” . . . 53
16 Sequence diagram for “Use Automated Mode” . . . 54
17 Sequence diagram for “Edit or Delete Own Paths” . . . 55
18 Sequence diagram for “Monitor System and Users” . . . 56
19 Sequence diagram for “Configure External Services and Parameters” . . . 57
20 Temporal trace for `show`: evolution of detected issues across steps (from *NewIssue* to *Pending*, *Confirmed*, or *Rejected*). . . . 66
21 Recommendation scenario for `show2`: one route query with multiple candidate paths and a single recommended candidate satisfying the maximal-score constraint. . . . 67

# List of Tables

1 Revision history of the RASD. . . . 10
2 Use Case: Register as a User . . . 34
3 Use Case: Log in as a User . . . 35
4 Use Case: Record a Trip . . . 36
5 Use Case: Define a Path Manually . . . 37
6 Use Case: Review Detected Issues . . . 38
7 Use Case: Search Bike Paths Between Origin and Destination . . . 39
8 Use Case: Manage Profile and Privacy Settings . . . 40
9 Use Case: View Trip History and Statistics . . . 41
10 Use Case: Use Automated Mode . . . 42
11 Use Case: Edit or Delete Own Paths . . . 43
12 Use Case: Monitor System and Users . . . 44
13 Use Case: Configure External Services and Parameters . . . 45
14 Mapping of Goals to Requirements and Domain Assumptions . . . 58

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

# 1 Introduction

## 1.1 Purpose

Urban cyclists often find it difficult to identify **safe, comfortable, and efficient routes** in their city. Information about bike lanes, traffic intensity, road quality, and obstacles (such as potholes or damaged surfaces) is usually incomplete, outdated, or scattered across many sources. At the same time, cyclists continuously generate valuable information while riding, such as GPS traces, speed profiles, and movements of their mobile devices, which typically remain unused. As a consequence, planning a pleasant and safe bike trip often relies on personal experience and trial-and-error, which may discourage people from using bikes as a regular means of transportation.

The **Best Bike Paths (BBP)** project aims to address these issues by providing a software system that helps cyclists **collect, maintain, and exploit an inventory of bike paths**. The system supports riders in recording their trips, annotating the quality of the paths they use, and reusing this knowledge to choose suitable routes between an origin and a destination. The ultimate purpose of BBP is to **enable cyclists to plan and perform safer and more effective bike trips** by relying on up-to-date, community-contributed information about bike paths. The purpose of the project is articulated into the following goals:

- **[G1 – Trip Recording]**

  Allow registered users to record their bike trips through a mobile device, capturing GPS traces and timestamps that describe the travelled route.

- **[G2 – Trip Statistics and History]**

  Provide users with statistics for each recorded trip (e.g., distance, duration, average speed) and maintain a personal history of their cycling activities, optionally enriched with relevant meteorological information, when available, from external services.

- **[G3 – Manual Path Definition]**

  Enable registered users to manually define bike paths by selecting streets or segments and associating them with properties such as path status (e.g., *optimal*, *medium*, *sufficient*, *requires maintenance*) and known obstacles (e.g., potholes, hazardous segments).

- **[G4 – Automated Path Data Acquisition]**

  Support an automated mode in which the system, while the user is biking, collects GPS data and sensor readings (accelerometer and gyroscope) to reconstruct the followed path and detect potential issues affecting path quality.

- **[G5 – Confirmation of Inferred Information]**

  Provide mechanisms for users to review, confirm, or correct the information inferred in automated mode (e.g., detected potholes), ensuring that only validated data becomes publishable.

- **[G6 – Path Inventory and Publication]**

  Maintain a structured inventory of bike paths and their associated information, and allow their owners to mark data as publishable so that it becomes available to all users of the system.

- **[G7 – Path Search and Visualization]**

  Allow users, including unregistered users, to request bike paths between a chosen origin and destination and visualize one or more candidate routes on a map.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

- **[G8 – Path Scoring and Recommendation]**

  Compute a score for each candidate path based on its status, obstacles, and effectiveness in reaching the destination from the origin, and use this score to rank alternative paths so that cyclists can easily choose the most suitable one.

Together, these goals define the purpose of BBP: **to turn cyclists' trips and feedback into actionable knowledge that improves the safety and quality of future bike routes.**

## 1.2 Scope

In the context of the *Best Bike Paths (BBP)* system, the relevant phenomena include:

### 1.2.1 World Phenomena

- **[WP1]**: Cyclists ride their bikes along existing streets and bike paths in the city, performing trips of varying duration and difficulty.
- **[WP2]**: The physical road infrastructure (streets, bike lanes, intersections) has properties such as surface quality, presence of potholes, and traffic intensity that may change over time.
- **[WP3]**: Weather conditions (e.g., rain, wind, temperature) influence the safety and comfort of cycling on given paths.
- **[WP4]:** Cyclists visually identify the status of the bike path (e.g., optimal, medium, sufficient).
- **[WP5]**: Cyclists plan journeys between an origin and a destination according to their own preferences (e.g., safety, comfort, travel time).
- **[WP6]**: Cyclists gain experience about the quality of the paths they use and may be willing to share this knowledge with others.

### 1.2.2 Shared Phenomena

World-Controlled Shared Phenomena:

- **[SP1]**: Users register to BBP, authenticate themselves, and manage their personal accounts through the application.
- **[SP2]**: Registered users start and stop the recording of their bike trips using their mobile devices.
- **[SP3]**: Users enable or disable the automated acquisition mode, granting or revoking permission for BBP to use location and motion sensors while they ride.
- **[SP4]**: Registered users manually define bike paths by selecting streets or segments in the application and assigning them a status and known obstacles.
- **[SP5]**: Users review, confirm, or correct information about path issues (e.g., potholes) that BBP has inferred from sensor data before this information becomes publishable.
- **[SP6]**: Users specify an origin and a destination and submit route queries to BBP.
- **[SP7]**: Users inspect the paths displayed by BBP on the map, compare alternatives, and choose which path to follow for their trip.

Machine-Controlled Shared Phenomena:

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

- **[SP8]**: BBP records trip data (GPS positions, timestamps) and stores it as part of the user's trip history.
- **[SP9]**: BBP computes and presents statistics about each recorded trip, such as distance, duration, average speed, and similar indicators, possibly enriched with weather information obtained from external services.
- **[SP10]**: BBP reconstructs the path followed during a trip from GPS data and analyzes motion sensor readings to detect potential obstacles or surface problems.
- **[SP11]:** The system prompts the user to confirm or reject potential obstacles detected during the ride.
- **[SP12]**: BBP updates the inventory of bike paths with manually inserted or confirmed information, including path status and obstacles, and marks data as publishable or private according to the owner's choices.
- **[SP13]**: BBP processes a user's origin–destination query, selects candidate bike paths from the inventory, computes a score for each path based on its quality and effectiveness, and ranks them accordingly.
- **[SP14]**: BBP visualizes the selected paths on a map, together with their scores and relevant details, so that users can evaluate them.
- **[SP15]**: BBP enforces access control and privacy rules when returning trip and path information, exposing only what each user is allowed to see.
- **[SP16]:** The system sends requests to external map/weather APIs to retrieve necessary environmental context.

## 1.3 Definitions, Acronyms, Abbreviations

This section introduces the terminology used throughout the document. Its aim is to avoid ambiguity and to ensure consistent usage of terms.

### 1.3.1 Definitions

- **Bike Path:** A route intended to be suitable for bicycles, either because a proper bike lane exists or because traffic conditions (e.g., low car density, compatible speed limits) make the road appropriate for cycling.
- **Path Segment:** A portion of a bike path corresponding, for example, to a single street or a continuous stretch of roadway between two intersections.
- **Trip:** A cycling activity performed by a registered user and recorded by BBP, including the start and end times of the activity and associated statistics such as total distance and average speed.
- **Trip Point (GPS Sample):** A single geolocated point acquired during a trip, consisting of a timestamp and coordinates (latitude and longitude). Trip points form the sequence used by BBP to reconstruct the followed path.
- **Trip Statistics:** Quantitative metrics computed by BBP for a trip, such as total distance, duration, elevation trends, and average speed.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

- **Meteorological Information / Weather Condition:** Weather-related data associated with a trip (e.g., temperature, wind speed, precipitation, general weather conditions) retrieved from an external service.

- **Obstacle:** A relevant issue affecting a bike path, such as a pothole, rough surface, or other road-surface irregularity. Obstacles may be manually reported by users or created automatically after the confirmation of detected issues.

- **Detected Issue:** A potential obstacle inferred automatically by BBP from device sensor data (e.g., accelerometer, gyroscope) acquired during a trip. Each detected issue includes a type, location, and confidence level and must be confirmed, corrected, or rejected by the user.

- **Path Status:** A qualitative evaluation of the overall condition of a bike path (e.g., optimal, medium, sufficient, requires maintenance).

- **Manual Mode:** A mode in which registered users explicitly insert information about bike paths, selecting the streets composing the path and setting their status and known obstacles.

- **Automated Mode:** A mode in which BBP acquires data automatically from the user's mobile device while they are biking. The system infers that the user is biking based on speed, reconstructs the followed path using GPS data, and detects possible obstacles using accelerometer and gyroscope measurements.

- **Confirmation Phase:** A phase following automated acquisition in which the user is asked to confirm, correct, or discard issues inferred by BBP before this information becomes available to the community.

- **Publishable Information:** Information about bike paths provided to BBP (either manually or automatically) that the owner chooses to make available to all users of the system.

- **Registered User:** A user who has an account on BBP and can record trips, insert information about bike paths, confirm automatically detected issues, and publish information.

- **Non-Registered User:** A user who accesses BBP without logging in and can only request the visualization of bike paths between an origin and a destination.

- **Origin:** The starting geographic point of a route specified by the user in a route search query.

- **Destination:** The target geographic point of a route specified by the user in a route search query.

- **Route Query:** A request made by a user to BBP specifying an origin and a destination for which the system must visualize one or more bike paths.

- **Candidate Path:** A bike path evaluated by BBP in response to a route query. For each candidate path, BBP computes a score and displays it as one of the possible routes.

- **Path Score:** A numerical or ordinal value computed by BBP for a bike path based on the path's status, presence of obstacles, and overall effectiveness in reaching the destination from the given origin.

- **Connected Segments:** Two PathSegments are connected if they share a common endpoint (intersection). In a continuous route, the end of one segment matches the start of the next.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

### 1.3.2 Acronyms

- **BBP**: Best Bike Paths.
- **GPS**: Global Positioning System.
- **RASD**: Requirements Analysis and Specification Document.
- **UI**: User Interface.
- **API**: Application Programming Interface.

The acronyms UI and API are used in later sections of this document to describe external interfaces and are therefore listed here for completeness.

### 1.3.3 Abbreviations

- **DB**: Database.
- **ID**: Identifier.
- **Avg.**: Average (e.g., Avg. speed).
- **Min.**: Minimum.
- **Max.**: Maximum.

## 1.4 Revision History

| Version | Date | Author(s) | Description |
|---|---|---|---|
| 0.1 | 2025-11-22 | Hamidreza Khodadadi, Mohammad Rashidkhan | Joint preparation of Introduction draft, Purpose, and Scope sections. |
| 0.2 | 2025-11-23 | Hamidreza Khodadadi, Mohammad Rashidkhan | Co-definition of terminology, Acronyms, Abbreviations, References, and Document Structure. |
| 0.3 | 2025-11-28 | Hamidreza Khodadadi, Mohammad Rashidkhan | Collaborative drafting of Overall Description and Product Perspective. |
| 0.4 | 2025-11-30 | Hamidreza Khodadadi, Mohammad Rashidkhan | Joint specification of Product Functions, Domain Assumptions, Dependencies, and Constraints. |
| 0.5 | 2025-12-01 | Hamidreza Khodadadi, Mohammad Rashidkhan | Drafting Specific Requirements and External Interface specifications. |
| 0.6 | 2025-12-05 | Hamidreza Khodadadi, Mohammad Rashidkhan | Refining and completing the Functional Requirements section. |
| 0.8 | 2025-12-14 | Hamidreza Khodadadi, Mohammad Rashidkhan | Development and verification of Formal Analysis (Alloy). |
| 1.0 | 2025-12-23 | Hamidreza Khodadadi, Mohammad Rashidkhan | Final review, formatting, and release. |

Table 1: Revision history of the RASD.

## 1.5 Reference Documents

This document is based on the following material:

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

- The official RASD assignment specification for the Software Engineering 2 course, academic year 2025–2026, containing the problem definition “Best Bike Paths”.
- Slides and notes from the Software Engineering 2 course concerning requirements engineering and the preparation of RASD documents.

## 1.6 Document Structure

- **Section 1 – Introduction**: presents the purpose and scope of BBP, defines key terminology (definitions, acronyms, abbreviations), and reports revision history, reference documents, and the structure of this RASD.
- **Section 2 – Overall Description**: provides a high-level view of the BBP system, including the product perspective, main functions, user characteristics, assumptions, dependencies, and constraints.
- **Section 3 – Specific Requirements**: details the functional and non-functional requirements of BBP, including external interface requirements, use cases and related diagrams, performance requirements, design constraints, and software system attributes.
- **Section 4 – Formal Analysis Using Alloy**: presents the formal model of selected aspects of BBP and the results of analyses performed using the Alloy tool.
- **Section 5 – Effort Spent**: summarizes the amount of work carried out by each group member for the preparation of this document.
- **Section 6 – References**: lists all external sources referenced in this RASD.

## 1.7 Use of Generative AI Tools

To support the preparation and refinement of this document, we used Generative AI tools during the development process. This subsection describes the tool(s) used, the inputs provided, the outputs obtained, and the steps taken to verify and integrate the results.

### 1.7.1 Tool(s) Used

- ChatGPT

### 1.7.2 Inputs Provided

1. Requests to suggest edits and improve coherence and cohesion across the document, particularly by identifying inconsistencies and enforcing consistent terminology (e.g., using the same term for the same concept throughout the document).
2. Requests to assist with creating diagram specifications using PlantUML (e.g., guidance and code generation for class diagrams and state diagrams).

### 1.7.3 Outputs Obtained

- Suggested corrections and improvements to ensure the text is coherent, cohesive, and consistent in terminology.
- More formal and technical language where appropriate.
- PlantUML diagram code and recommendations to support the production of diagrams (e.g., class diagrams and state diagrams).

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

### 1.7.4 Verification, Refinement, and Integration

- We reviewed the generated suggestions to ensure they align with the project scope and requirements.
- For PlantUML outputs, we executed the generated code to validate that the diagrams compiled and rendered correctly.
- When issues were identified (e.g., incorrect assumptions, inconsistent terminology, or PlantUML syntax problems), we manually refined the text or modified the diagram code until correct results were obtained.
- Only verified and project-consistent outputs were integrated into the final version of the document.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

# 2 Overall Description

This section provides a high-level overview of the Best Bike Paths (BBP) system. It outlines the context in which BBP operates and summarizes its overall perspective, key functions, intended users, and the main assumptions, dependencies, and constraints that shape the solution.

## 2.1 Product Perspective

### 2.1.1 Scenarios

**A Cyclist Records a Trip**
Luca, a regular cyclist who often rides across the city, opens the BBP mobile application before starting his commute. After logging in, he taps the *Start Trip* button. BBP activates Luca's GPS and begins collecting his location data every few seconds. As he rides, the application also monitors the readings from his phone's accelerometer and gyroscope, registering significant vertical or lateral movements that may indicate bumps or potholes on the road.

During the trip, Luca checks the live dashboard, which shows elapsed time, current speed, and total distance covered. At some point, he rides through a rough segment of road; the phone vibrates noticeably, and BBP logs this sensor peak as a potential obstacle.

After reaching his destination, Luca taps *Stop Trip*. The system stores the trip in his personal history and enriches it with weather information (temperature, wind, rain intensity) retrieved from an external weather service. Later, Luca reviews detailed trip statistics such as distance, duration, and average speed. The final GPS position recorded when he stopped the trip is considered the destination of that trip.

**A Cyclist Defines a Path Manually**
Maria wants to contribute to the community by documenting a bike-friendly route she frequently uses to reach her university. She logs into BBP and navigates to the *Manual Path Submission* section.

Here, she is presented with an interactive map. Maria selects the streets she uses, adding each segment to the path. BBP guides her to build a continuous route: she can only add a new segment if it connects to the last selected segment; otherwise the system prompts her to choose a connecting street. For each selected segment, she assigns a path status such as:

- "Optimal" for recently renewed bike lanes,
- "Medium" for regular asphalt,
- "Sufficient" for a street where traffic is a bit heavier.

Along the route, Maria also marks relevant obstacles: a pothole near a particular intersection, a slippery segment on a bridge when wet, and a construction zone that temporarily narrows one of the lanes. After reviewing her inputs, she marks the path information as *publishable*, making it available to the rest of the BBP community.

**A Cyclist Uses Automated Mode to Gather Path Information**
Alberto enables *Automated Mode* in the BBP app because he enjoys cycling through different parts of the city and wants BBP to automatically gather useful data from his rides. Once activated, the system continuously collects GPS positions, speed, and device motion signals, inferring Alberto's followed path.

During one of his rides, the accelerometer captures several sharp vertical movements when Alberto rides on cobblestones. BBP interprets these readings as possible road surface issues. However, the system knows such detections may include false positives; for example, the cyclist might simply drop the phone or hit a curb.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

After the ride, BBP stores the detected issues (DetectedIssues) and notifies Alberto that they are ready for review in the *Review Detected Issues* screen. The issues remain unpublishable until Alberto validates them. For each detection, Alberto can:

- confirm it as a real pothole or bump,
- correct the classification (e.g., "not a pothole, just cobblestones"), or
- reject the detection entirely.

Only after confirmation does BBP add the validated information to the path inventory. Alberto appreciates this workflow because it prevents incorrect data from being published.

**A User Searches for Bike Paths Between an Origin and a Destination**
Elena, who is planning to ride from her home to a park across town, opens the BBP app without logging in. She goes to the *Find Path* feature and enters her origin and destination. BBP queries its inventory of publishable paths contributed by the community.

The system identifies several potential paths connecting the two points. Each path has an associated score, computed using:

- its overall status,
- the number and severity of obstacles along it,
- and its effectiveness in reaching the destination (for example, length and directness).

BBP then presents Elena with a set of routes on the map. The highest-scoring route is highlighted, while alternatives are also shown. Each route displays a summary of:

- expected distance and time,
- number of obstacles,
- whether segments are optimal, medium, or require maintenance.

Elena selects the highest-scoring route and starts navigating it using the map view provided by the app.

**A Cyclist Reviews and Publishes Automatically Detected Information**
Sara rides daily to work using Automated Mode. After completing a trip in Automated Mode, Sara opens the Review Detected Issues screen. BBP highlights a candidate issue that has been detected repeatedly over the past few trips and shows an aggregated view. The system groups these observations and prompts Sara to validate them.
She opens the *Review Detected Issues* screen, which lists, for each candidate issue:

- location of the issue,
- frequency of detection,
- suggested issue category (e.g., pothole, uneven pavement).

Sara inspects the area on the map and confirms that the issue is indeed a pothole, which has been getting worse over the weeks. She marks the detection as correct and chooses to make the information publishable. The corresponding path segment's status is updated to "requires maintenance", improving BBP's accuracy for other users.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

### 2.1.2 Domain-level Class Diagram

The domain-level class diagram models the key entities and relationships that characterize the Best Bike Paths (BBP) application domain. It represents central concepts such as **Cyclist**, **Trip**, **Path**, **PathSegment**, **Obstacle**, **DetectedIssue**, and **RouteQuery**, along with their relevant attributes and the interactions among them.

The model captures how trips are recorded and reconstructed from GPS samples, how users manually or automatically contribute information about bike paths, how obstacles emerge and become publishable through user confirmation, and how BBP retrieves and ranks candidate paths for route queries. By focusing on world and shared phenomena rather than implementation details, the diagram provides a conceptual overview of how information flows within the domain.

A **Trip** represents a cycling activity recorded by a registered user. Each trip consists of a sequence of **TripPoints**, which are individual GPS samples collected during the ride. After the recording ends, BBP reconstructs the travelled route by geographically matching these TripPoints to the existing **PathSegments** in the inventory. This relationship is represented by the association indicating that a Trip may *traverse* several PathSegments.

During a trip, BBP may infer anomalies from accelerometer and gyroscope data, creating **DetectedIssues**. Each DetectedIssue is associated with exactly one Trip and is localized on exactly one PathSegment, reflecting where the potential obstacle occurred. After the ride, users must review the detected issues: confirmed or corrected ones result in the creation of new **Obstacles** on that segment, while rejected ones are discarded. Obstacles and paths can each be marked as *publishable*, determining the visibility of the information within the shared path inventory.

A **Path** is a route defined by a sequence of PathSegments, either manually specified by the user or derived from previously collected information. Each Path has a qualitative **status** selected from a finite set of values (e.g., optimal, medium, sufficient, requires maintenance). A Path includes at least one PathSegment, while each PathSegment may belong to one or more Paths.

Any user—registered or not—can request suggested bike routes by submitting a **RouteQuery**, specifying an origin and destination, each modeled as a **Location** value object. BBP evaluates candidate paths against the query and constructs **CandidatePath** entities, each referencing a Path and associated with a real-valued score computed from the path's condition, obstacles, and effectiveness in reaching the destination. Non-registered users are not explicitly represented in the diagram; their behavior is captured by RouteQuery instances without an associated Cyclist.

The following diagram provides a conceptual overview of the problem domain and avoids any implementation-specific detail.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

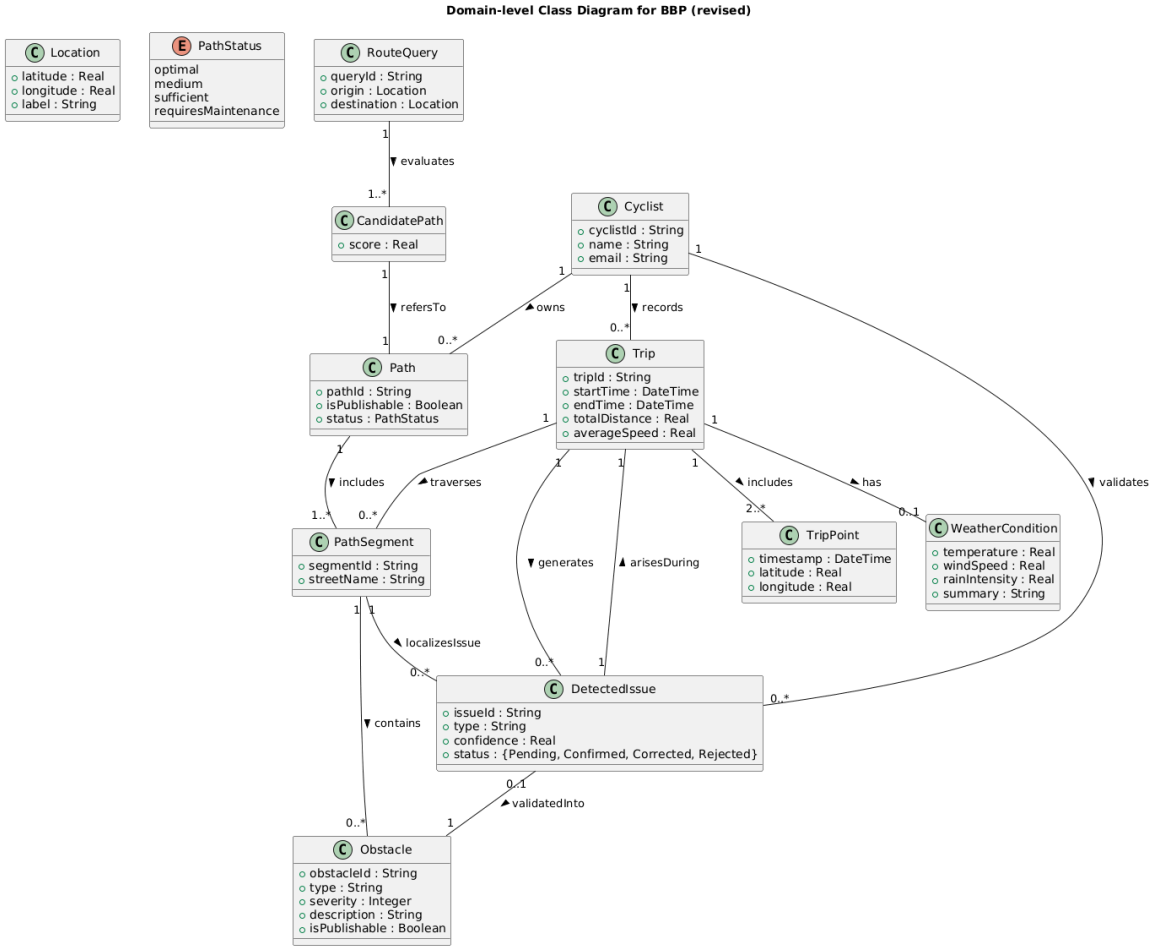



Figure 1: Domain class diagram

### 2.1.3 State Diagrams

State diagrams illustrate the dynamic behavior of key domain entities in BBP. They describe how Trips evolve during recording, how automatically DetectedIssues move through confirmation stages, and how Paths become publishable.

**Trip State Diagram**
This diagram represents the lifecycle of a recorded Trip. A trip begins when a cyclist starts recording. While the trip is ongoing, BBP collects GPS samples and sensor readings. When the cyclist stops recording, the system reconstructs the travelled route, computes statistics, optionally retrieves weather data, and stores the completed trip in the user's history.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

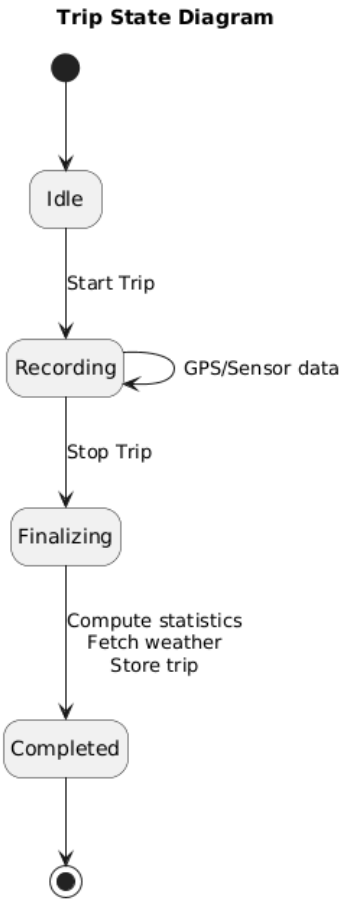



Figure 2: Trip State Diagram

**Detected Issue State Diagram**

This diagram models the lifecycle of a Detected Issue, that is, a potential obstacle inferred automatically from accelerometer and gyroscope anomalies during a trip. Since the system may produce false positives, each detected issue must go through a confirmation phase where the user validates, corrects, or rejects the inferred information. Confirmed or corrected issues become Obstacles; rejected ones are discarded.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

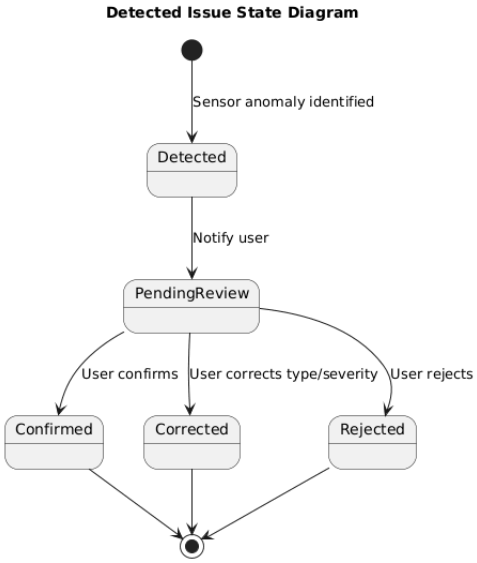



Figure 3: Detected Issue state Diagram

**Path State Diagram**
This diagram represents the lifecycle of a Path defined by a registered user. A path starts as a draft in manual mode, while the user selects segments and sets status. The path can then be saved as non-publishable (private), updated later, or made publishable so that all users can access it. Users may also update path information over time, leading the path back to the Editable state.

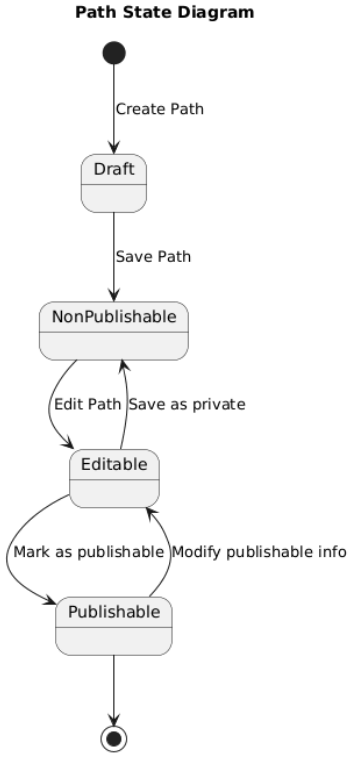



Figure 4: Path State Diagram

**Obstacle State Diagram**
This diagram represents the lifecycle of an Obstacle, i.e., a confirmed issue associated with a path seg-

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

ment. An obstacle can be created either manually by the user or automatically after the confirmation of a detected issue. When created, the obstacle is initially private to its owner. The owner can then decide whether to publish or unpublish it, controlling its visibility in the shared path inventory.

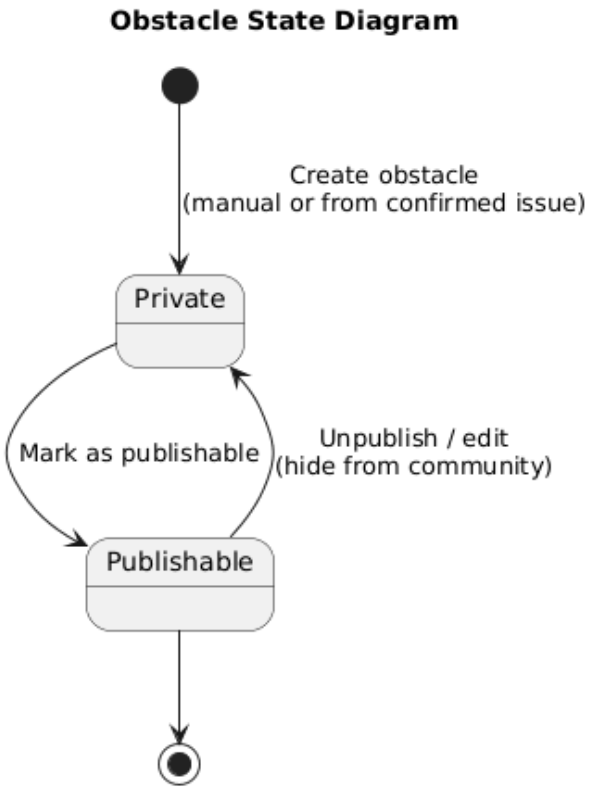



Figure 5: Obstacle State Diagram
Obstacle lifecycle from creation to publication.

**RouteQuery State Diagram**
This diagram represents the lifecycle of a **RouteQuery**, that is, a user request to visualize bike paths between an origin and a destination. A query is first composed by the user (setting origin and destination), then submitted to BBP for processing. The system retrieves candidate paths, computes their scores, and finally presents the ordered results on the map. After the answers are shown, the query reaches a completed state. Non-registered users issue queries in exactly the same way; they are not modeled explicitly here and are represented simply by RouteQuery instances without an associated Cyclist.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

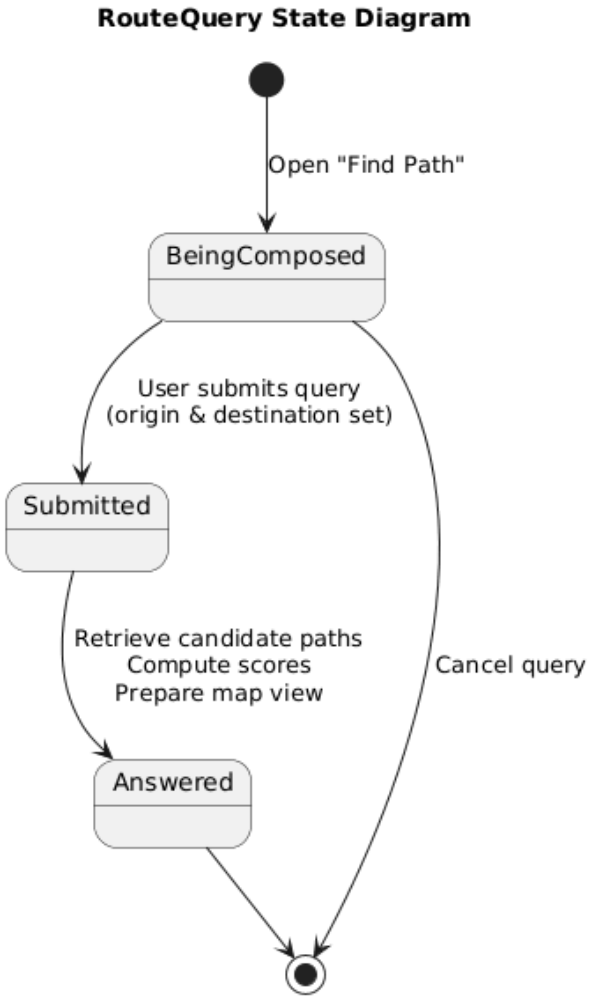



Figure 6: Route query lifecycle from composition to answer.

## 2.2 Product Functions

This section summarizes the main functions that BBP must provide in order to achieve the project goals defined in Section 1.1 and to support the scenarios in Section 2.1.

### 2.2.1 Trip Management

- **PF-T1 – Trip Recording**
  BBP shall allow a registered user to start and stop the recording of a bike trip from the mobile application. While recording, the system collects GPS samples and motion sensor data (accelerometer and gyroscope) at suitable intervals.

- **PF-T2 – Trip Reconstruction and Storage**
  After a trip is stopped, BBP shall reconstruct the travelled route from the collected GPS samples and store the trip in the user's personal history, together with its start/end times.

- **PF-T3 – Trip Statistics**
  For each recorded trip, BBP shall compute and store basic statistics such as total distance, duration, and average speed. These statistics shall be made available to the user in a trip-details view.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

- **PF-T4 – Weather Enrichment (When Available)**
  When an external weather service is reachable, BBP shall retrieve relevant meteorological information for the time and approximate area of the trip and associate it with the recorded trip.

- **PF-T5 – Trip History Browsing**
  BBP shall allow a registered user to browse their past trips, view trip details (including route and statistics), and select individual trips for inspection.

#### 2.2.2 Manual Path Information Management

- **PF-P1 – Manual Path Definition**
  BBP shall allow a registered user to define a bike path manually by selecting streets or segments on a map and composing them into a Path.

- **PF-P2 – Path Status Management**
  For each user-defined path, BBP shall allow the owner to set and update the overall path status (e.g., optimal, medium, sufficient, requires maintenance).

- **PF-P3 – Manual Obstacle Annotation**
  BBP shall allow a registered user to associate obstacles (e.g., potholes, rough surface, dangerous crossings) to specific path segments, including information such as type, severity, and a short description.

- **PF-P4 – Publishable Path Information**
  BBP shall allow the owner of a path to mark the path information as publishable, so that its geometry, status, and publishable obstacles become part of the shared inventory of bike paths accessible to all users.

#### 2.2.3 Automated Path Data Acquisition and Confirmation

- **PF-A1 – Automated Mode Activation**
  BBP shall allow a registered user to enable or disable an automated acquisition mode in which the system collects GPS and motion sensor data while the user is biking.

- **PF-A2 – Automatic Detection of Potential Issues**
  While automated mode is active and a trip is being recorded, BBP shall detect potential issues (DetectedIssues) based on sensor anomalies and associate them with the corresponding trip and path segment.

- **PF-A3 – Issue Review and Confirmation Phase**
  After a trip, BBP shall present the detected issues to the user and allow them to confirm, correct, or reject each issue.

- **PF-A4 – Conversion to Obstacles**
  For issues that are confirmed (possibly after correction), BBP shall create or update corresponding obstacles on the relevant path segments and, if the user chooses, mark them as publishable information

#### 2.2.4 Path Search, Visualization and Recommendation

- **PF-R1 – Route Query Submission**
  BBP shall allow any user, whether registered or not, to specify an origin and a destination and submit a route query.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

- **PF-R2 – Candidate Path Retrieval**
  In response to a route query, BBP shall retrieve zero (if none found, inform user) or more candidate paths between the origin and the destination from the inventory of publishable paths and segments.

- **PF-R3 – Path Scoring and Ranking**
  For each candidate path, BBP shall compute a score that depends on the path status, the presence and severity of publishable obstacles, and the effectiveness of the path in reaching the destination from the origin. BBP shall order the candidate paths according to this score.

- **PF-R4 – Map Visualization of Paths**
  BBP shall display the candidate paths on a map, together with basic summary information (e.g., path length and qualitative score), so that users can visually compare and select a preferred path.

- **PF-R5 – Path Details Visualization**
  BBP shall allow the user to inspect a selected candidate path in detail, including the list and position of publishable obstacles along the path and the distribution of path statuses on its segments.

#### 2.2.5 User and Access Management

- **PF-U1 – Registration and Authentication**
  BBP shall allow new users to register and existing users to authenticate, so that trips and owned path information can be associated with the correct Cyclist.

- **PF-U2 – Ownership and Visibility Control**
  BBP shall associate each path and piece of path information with its owner and enforce visibility rules so that only publishable information is visible to all users, while non-publishable information can be seen and modified only by its owner.

- **PF-U3 – Access for Non-Registered Users**
  BBP shall allow non-registered users to submit route queries and view publishable paths and obstacles but shall not allow them to record trips or create/modify path information.

### 2.3 User Characteristics

The BBP platform supports two primary types of end users: **registered cyclists** and **non-registered users**. In addition, **system administrators** manage technical and moderation tasks. Each user type has specific roles and characteristics when interacting with the platform.

#### 2.3.1 Registered Cyclist

A registered cyclist is an individual who creates an account on BBP to record trips and contribute information about bike paths. Registered cyclists can start and stop trip recording from the mobile application, allowing the system to collect GPS and sensor data (accelerometer and gyroscope) during their rides. They can review their trip history, inspect trip statistics, and, when available, see associated meteorological information.

Registered cyclists are also responsible for enriching the path inventory. In **manual mode**, they can define bike paths by selecting streets or segments on the map, assign path statuses, and annotate obstacles on specific segments. In **automated mode**, they allow BBP to detect potential issues from sensor data and later participate in the confirmation phase by confirming, correcting, or rejecting detected issues. Furthermore, registered cyclists can decide which paths and obstacles become publishable so that other users can benefit from the shared information.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

### 2.3.2 Non-Registered User

A non-registered user is any person who accesses BBP without creating or logging into an account. These users are primarily consumers of the information collected by the community. They can open the application or web interface, specify an origin and destination, and request bike paths between these points. BBP presents them with one or more candidate routes, visualized on a map and ranked according to their quality and effectiveness.

Non-registered users cannot record trips, define paths, annotate obstacles, or participate in the confirmation of automatically detected issues. Their interaction is limited to browsing publishable paths and using the platform as a decision-support tool for planning bike routes.

### 2.3.3 System Administrator

A system administrator is responsible for the technical operation and maintenance of BBP. Administrators do not normally interact with the platform as cyclists but instead manage system-level aspects such as user account administration, monitoring of service availability, and configuration of external services (e.g., map and weather providers).

When necessary, administrators may moderate or remove path and obstacle information that is clearly incorrect, abusive, or violates legal or community guidelines. They also oversee data retention policies and ensure that stored trip and path data is handled according to privacy and security requirements.

## 2.4 Domain Assumptions, Dependencies, and Constraints

### 2.4.1 Domain Assumptions

- **[DA1: Internet and Device Access]**
  All users (registered and non-registered) have access to the internet and to a GPS-enabled mobile device (e.g., smartphone) with accelerometer and gyroscope sensors, capable of running the BBP application.

- **[DA2: Accurate Location and Sensor Readings]**
  The GPS, accelerometer, and gyroscope readings provided by users' devices are sufficiently accurate and frequent to reconstruct trips and to detect potential road-surface issues (e.g., potholes) with acceptable precision.

- **[DA3: Honest Path Information and Feedback]**
  Users are expected to provide honest and reasonably accurate information when manually defining paths, annotating obstacles, and confirming or rejecting automatically detected issues.

- **[DA4: Stable Road Network]**
  The underlying road network (streets and intersections) does not change dramatically over short periods, so that paths and path segments remain meaningful for a reasonably long time.

- **[DA5: Reliable Map Data]**
  The external map provider used by BBP supplies correct and up-to-date geographic information (e.g., street geometries and names) that can be relied upon when defining paths and matching trips to path segments.

- **[DA6: External Weather Service Availability]**
  When BBP requests meteorological information for a trip, the external weather service is assumed to be reachable and to return accurate weather data for the requested time and area.

- **[DA7: Reasonable Trip Duration and Battery Level]**
  Users start trip recording with a sufficient battery level and do not perform trips that are unusually

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

long relative to their device's battery capacity, so that recording can run continuously without unexpected interruptions due to power loss.

- **[DA8: Single Rider per Device During Trip]**
  Each running instance of the BBP app is assumed to correspond to a single cyclist during a trip; devices are not shared among multiple cyclists while a trip is being recorded.

- **[DA9: Non-Malicious Intent]**
  The platform assumes that users do not deliberately generate fake trips, obstacles, or confirmations for the purpose of degrading the quality of the shared path information.

### 2.4.2 Dependencies

- **Map and Geocoding Services**
  BBP depends on external map and geocoding services to display maps, obtain street geometries, and associate GPS coordinates with streets and intersections when defining paths or reconstructing trips.

- **Weather Information Service**
  The system relies on an external weather provider to retrieve meteorological information (e.g., temperature, wind, precipitation) associated with recorded trips.

- **Cloud Hosting and Storage**
  BBP depends on a cloud-based infrastructure for hosting its backend services, storing user accounts, trips, paths, obstacles, and associated metadata, and for providing sufficient availability and scalability.

- **User Devices**
  The platform depends on compatible mobile devices (smartphones) equipped with GPS, accelerometer, and gyroscope sensors, and with a supported operating system version, for smooth interaction and data collection.

- **Notification / Messaging Services (if used)**
  If BBP sends push notifications or e-mails (e.g., reminders to review detected issues), it depends on third-party notification or e-mail services to deliver these messages to users.

### 2.4.3 Constraints

- **Supported Platforms**
  The initial release of BBP is intended to run on modern smartphone platforms (e.g., recent versions of Android and/or iOS). Other platforms (desktop browsers, legacy devices) are not initially supported.

- **Sensor and Sampling Limits**
  To preserve battery life and network usage, BBP must limit the frequency of GPS and sensor sampling and may batch or compress recorded data before transmission to the backend.

- **Data Storage and Retention**
  The system must store trip and path data in a way that respects reasonable storage limits. Very old or low-value raw sensor data may be aggregated or archived according to policies defined at design time.

- **Privacy and Data Protection**
  The collection and processing of location and sensor data must comply with applicable privacy

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

regulations. Personal data (including trip histories) must be stored securely, and access to it must be restricted based on user identity and ownership.

- **Language Support**
  The initial version of BBP will support only one primary language (e.g., English). Additional languages may be considered as future enhancements.

- **Development Timeline**
  The system must be designed and implemented within the schedule defined for the course project, which constrains the amount of functionality and level of detail that can be implemented in the first release.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

# 3 Specific Requirements

## 3.1 External Interface Requirements

This section describes how the Best Bike Paths (BBP) system interacts with users, devices, other software systems, and communication channels.

### 3.1.1 User Interfaces

The BBP system provides different user interfaces tailored to the main categories of users: registered cyclists, non-registered users, and system administrators.

**Cyclist Interface (mobile application).** The mobile application is the primary interface for registered users while riding and when managing their personal cycling data. The main views are:

- **Login / Registration View**: allows a user to create an account or log in using email and password. It also provides access to password reset and to the option of using the application as a guest (limited features).
- **Home / Dashboard View**: displays a summary of the user's recent activity (e.g., last trips, total distance travelled), shortcuts to start a new recording, to open the trip history, and to define or edit paths.
- **Trip Recording View**: shows a map centred on the current position, current speed, elapsed time, and distance. It provides controls to start and stop the recording. A status indicator reports whether automated mode (sensor-based obstacle detection) is active.
- **Trip History and Details View**: lists previously recorded trips with basic data (date, distance, duration). Selecting a trip opens a detailed page with the reconstructed path on the map, statistics (distance, duration, average speed), and, when available, meteorological information associated with the trip.
- **Manual Path Definition View**: provides an interactive map where the cyclist selects streets or segments to compose a bike path. For each segment the user can set the path status (e.g., optimal, medium, sufficient, requires maintenance) and annotate obstacles (potholes, rough surfaces, construction zones, etc.). The view allows the user to decide whether the information is private or publishable. BBP guides the user to build a continuous route. After the first segment is selected, BBP shall allow the user to add a new segment only if it is connected to the last selected segment (i.e., the end intersection of the current segment matches the start intersection of the next one). If the user selects a non-connecting segment, BBP shall prompt the user to choose a connecting street segment.
- **Detected Issues Review View**: lists issues detected automatically from accelerometer and gyroscope data during trips in automated mode. For each issue the user can see its location and type and can confirm it, correct its attributes, or reject it. Only confirmed issues can become part of the shared inventory.
- **Path Search and Visualization View**: allows both registered and non-registered users to specify an origin and a destination either by selecting them on the map or by entering textual addresses. The system displays one or more candidate paths on the map, each with a summary (distance, estimated travel time, path score, number and severity of obstacles). Users can compare these paths and choose one to follow.
- **Settings View**: lets users configure privacy options (default visibility of their contributions), manage permissions for location and motion sensors, and request deletion of their account and data.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

**Guest Interface (mobile and web).** Non-registered users can access a reduced set of features without authentication:

- access to the *Path Search and Visualization* view to look for bike paths between an origin and a destination;
- read-only access to path summaries and statistics;
- a prompt informing them that recording trips, defining paths, and confirming issues require registration.

**Administrator Interface (web).** System administrators access BBP through a web-based interface. Main functions include:

- monitoring system status and resource usage;
- viewing aggregated statistics about trips, paths, and obstacles;
- managing user accounts (e.g., disabling accounts in case of misuse);
- configuring connections to external services such as map and weather providers.

### 3.1.2 Hardware Interfaces

BBP interacts with the following hardware components:

- **User Mobile Devices**: smartphones or tablets used by cyclists must provide:
  - a GPS receiver (or equivalent location hardware) to acquire the sequence of positions during a trip;
  - an accelerometer and a gyroscope to detect significant movements used for identifying potential obstacles;
  - sufficient storage to hold temporary trip data when no network connection is available.

  The mobile application accesses these sensors via the standard APIs of the operating system and requires explicit user permission.
- **User Desktop / Laptop Computers**: used mainly to access the web interface. They must support a modern web browser and typical input devices (keyboard, mouse, or touchpad). No additional hardware is required.
- **Backend Servers**: one or more servers host the BBP backend components and databases. The RASD does not prescribe specific hardware, but the infrastructure must provide persistent storage and sufficient processing power to handle the expected user load.

### 3.1.3 Software Interfaces

BBP depends on several software interfaces:

- **Mobile Operating Systems**: the mobile application runs on major smartphone platforms (e.g., Android, iOS) and uses their APIs for:
  - location services (GPS and, when available, network-based positioning);
  - access to motion sensors (accelerometer, gyroscope);

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

  - notification delivery;
  - handling of background execution and power-saving modes.

- **Web Browsers**: the web interface is implemented as a responsive web application and must operate correctly on recent versions of main browsers that support HTML5, CSS3, and modern JavaScript.
- **Backend REST API**: both mobile and web clients communicate with the server through a RESTful HTTP API. The API exposes operations for user authentication, trip upload, retrieval of trip histories, creation and modification of paths, search of paths between an origin and a destination, and management of detected issues. Requests and responses are encoded in a structured format (e.g., JSON).
- **Database Management System**: the backend uses a DBMS to persist users, trips, paths, obstacles, and system configuration data. All access to the DBMS is mediated by the backend application; clients never interact with the database directly.
- **External Map Service**: BBP integrates with a third-party map service through its API to display maps, convert addresses to coordinates, and obtain a base street network on which bike paths are defined. The service is expected to expose HTTPS endpoints and return data in machine-readable formats.
- **External Weather Service**: to enrich trips with meteorological information, the backend calls an external weather API using trip time and location. The service returns information such as temperature, precipitation, and wind speed.

### 3.1.4 Communication Interfaces

The communication channels used by BBP must satisfy the following requirements:

- **Network Protocols**: all communication between clients, backend, and external services uses HTTP or HTTPS. HTTPS is mandatory whenever authentication credentials or personal data are transmitted.
- **Data Formats**: the system uses standard formats such as JSON for data exchanged via the REST API. Map tiles and similar resources follow the formats defined by the selected map provider.
- **Notifications**: when necessary (e.g., to confirm that a trip has been saved or to remind the user to review detected issues), the backend uses platform-specific push-notification services or other mechanisms provided by the mobile operating system.
- **Offline Operation and Synchronization**: if network connectivity is not available during a trip, the mobile application stores recorded data locally. Once the connection is restored, the data are sent to the backend through the REST API. The server is responsible for resolving potential conflicts (for example, if a path is edited from two devices).
- **Security**: all communication channels carrying personal information or detailed trip data must use TLS. Expired or invalid certificates must be rejected, and authentication tokens must be protected during transmission.

## 3.2 Functional Requirements

This section specifies what the Best Bike Paths (BBP) system shall do. The requirements are grouped according to the main functionalities identified in the goals and scenarios of Section **??**. Each requirement is identified by a unique label of the form **[R*i*]**.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

### 3.2.1 List of Functional Requirements

**User and account management**

- **[R1 – User Registration]**
  The system shall allow a cyclist to create a personal account by providing at least an email address and a password. The system shall verify that the email address is not already associated with an existing account.

- **[R2 – User Authentication]**
  The system shall allow a registered user to authenticate by entering valid credentials. After successful authentication the user shall access all features reserved to registered users.

- **[R3 – Guest Access]**
  The system shall allow non-registered users to access the path search and visualization functionality without authentication. Guests shall not be able to record trips, define paths, or confirm automatically detected issues.

- **[R4 – Profile Management]**
  The system shall allow a registered user to view and update their profile information (e.g., name, optional preferences related to path selection) and to change their password.

- **[R5 – Privacy and Publication Settings]**
  The system shall allow a registered user to configure default publication preferences for the information they contribute (e.g., whether newly created paths are private by default or publishable).

**Trip recording and statistics**

- **[R6 – Start Trip Recording]**
  The system shall allow a registered user to explicitly start the recording of a trip from the mobile application.

- **[R7 – Stop Trip Recording]**
  The system shall allow the user to stop the recording of an ongoing trip. When recording is stopped, the system shall store the trip in the user's trip history.

- **[R8 – Trip Data Acquisition]**
  While a trip is being recorded, the system shall periodically acquire and store GPS samples (position and timestamp). If available, the system shall also store instantaneous speed obtained from the device or computed from positions.

- **[R9 – Trip Statistics Computation]**
  For each recorded trip the system shall compute statistics including at least total distance, duration, and average speed.

- **[R10 – Trip History Management]**
  The system shall maintain a history of recorded trips for each registered user and shall allow the user to browse this history and view the details and statistics of any past trip.

- **[R11 – Weather Information Retrieval]**
  For a recorded trip, the system shall be able to query an external weather service using the time interval and the geographic area of the trip and shall store the obtained meteorological information together with the trip when such a service is available.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

**Manual path definition and management**

- **[R12 – Manual Path Creation]**
  The system shall allow a registered user to create a bike path manually by selecting streets or segments on an interactive map. After the first segment is selected, the system shall allow adding a segment only if it is connected to the last selected segment. Otherwise, the system shall reject the selection and prompt the user to select a connecting segment.

- **[R13 – Path Status Annotation]**
  For a manually defined path, the system shall allow the user to assign a qualitative status to each segment or to the path as a whole (e.g., *optimal*, *medium*, *sufficient*, *requires maintenance*).

- **[R14 – Obstacle Annotation]**
  For a manually defined path, the system shall allow the user to record obstacles such as potholes or rough surfaces by specifying their approximate location and type.

- **[R15 – Path Publication]**
  The system shall allow the user to mark a manually defined path and its annotations as *publishable* or *private*. Only publishable information shall be considered for path search by other users.

- **[R16 – Path Editing and Deletion]**
  The system shall allow the owner of a path to modify or delete the path and its associated annotations at any time.

**Automated mode and issue detection**

- **[R17 – Automated Mode Activation]**
  The system shall allow a registered user to activate an automated acquisition mode in which the application continuously observes the user's speed and sensor data.

- **[R18 – Biking Activity Detection]**
  In automated mode, the system shall infer that the user is biking when their motion satisfies configurable conditions (e.g., speed above a threshold for a minimum period). Only during inferred biking intervals shall the system treat collected data as candidate bike trips.

- **[R19 – Automated Path Reconstruction]**
  When the user is inferred to be biking, the system shall use collected GPS samples to reconstruct the followed path as a sequence of segments.

- **[R20 – Candidate Issue Detection]**
  While the user is biking, the system shall analyze accelerometer and gyroscope data to identify significant events that may correspond to obstacles (e.g., potholes, rough surface) and shall create candidate issues at the corresponding locations.

- **[R21 – Offline Data Buffering]**
  If no network connection is available while data is being collected, the system shall buffer the acquired data locally and transmit it to the backend when connectivity is restored.

**Confirmation of inferred information**

- **[R22 – Presentation of Detected Issues]**
  After a trip with automated acquisition, the system shall present the list of candidate issues detected during that trip to the user, including their location and proposed type.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

- **[R23 – Issue Confirmation]**
  The system shall allow the user to confirm that a candidate issue corresponds to a real obstacle. In this case the issue becomes a validated obstacle associated with the corresponding path.

- **[R24 – Issue Correction]**
  The system shall allow the user to correct the attributes of a candidate issue (e.g., location or type) before validating it.

- **[R25 – Issue Rejection]**
  The system shall allow the user to reject a candidate issue that does not correspond to an actual obstacle, in which case the issue shall not be stored in the shared inventory.

- **[R26 – Publication of Confirmed Issues]**
  Only issues that have been confirmed (possibly after correction) by their owner and marked as publishable according to the user's settings shall be made available to other users of the system.

**Path inventory and publication**

- **[R27 – Path Inventory Maintenance]**
  The system shall maintain an inventory of all paths known to BBP, including paths derived from trip data and paths manually defined by users, together with their status and obstacles.

- **[R28 – Ownership and Access Control]**
  For each piece of path information (path, status, obstacle) the system shall store its owner and publication flag and shall ensure that only the owner can modify or delete it.

- **[R29 – Use of Publishable Information]**
  When answering path search queries, the system shall only rely on path information that is marked as publishable by its owner.

**Path search, scoring, and visualization**

- **[R30 – Origin–Destination Query]**
  The system shall allow any user (registered or guest) to submit a path search query by specifying an origin and a destination.

- **[R31 – Candidate Path Selection]**
  For a valid query the system shall determine zero or more candidate bike paths connecting the origin and the destination by using the inventory of publishable paths and, when necessary, information from the underlying street network. If no candidate path is found, the system shall notify the user and suggest adjusting origin/destination.

- **[R32 – Path Score Computation]**
  For each candidate path the system shall compute a score that depends at least on the path status and the presence and severity of obstacles, and on the effectiveness of the path in reaching the destination from the origin (e.g., length or detour).

- **[R33 – Path Ranking]**
  The system shall rank candidate paths based on their score and shall use this ranking when presenting the results to the user.

- **[R34 – Path Visualization]**
  The system shall visualize each candidate path on a map, using a graphical representation that clearly distinguishes different paths and highlights segments with low status or many obstacles.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

- **[R35 – Path Summary Presentation]**
  For each candidate path the system shall present a summary including at least its length, an estimate of travel time, its overall score, and the number of obstacles along the path.

### 3.2.2 Use Case Diagram

Figure 7 illustrates the main use cases of the BBP system and their relationships with the actors *Registered Cyclist*, *Guest User*, and *System Administrator*.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

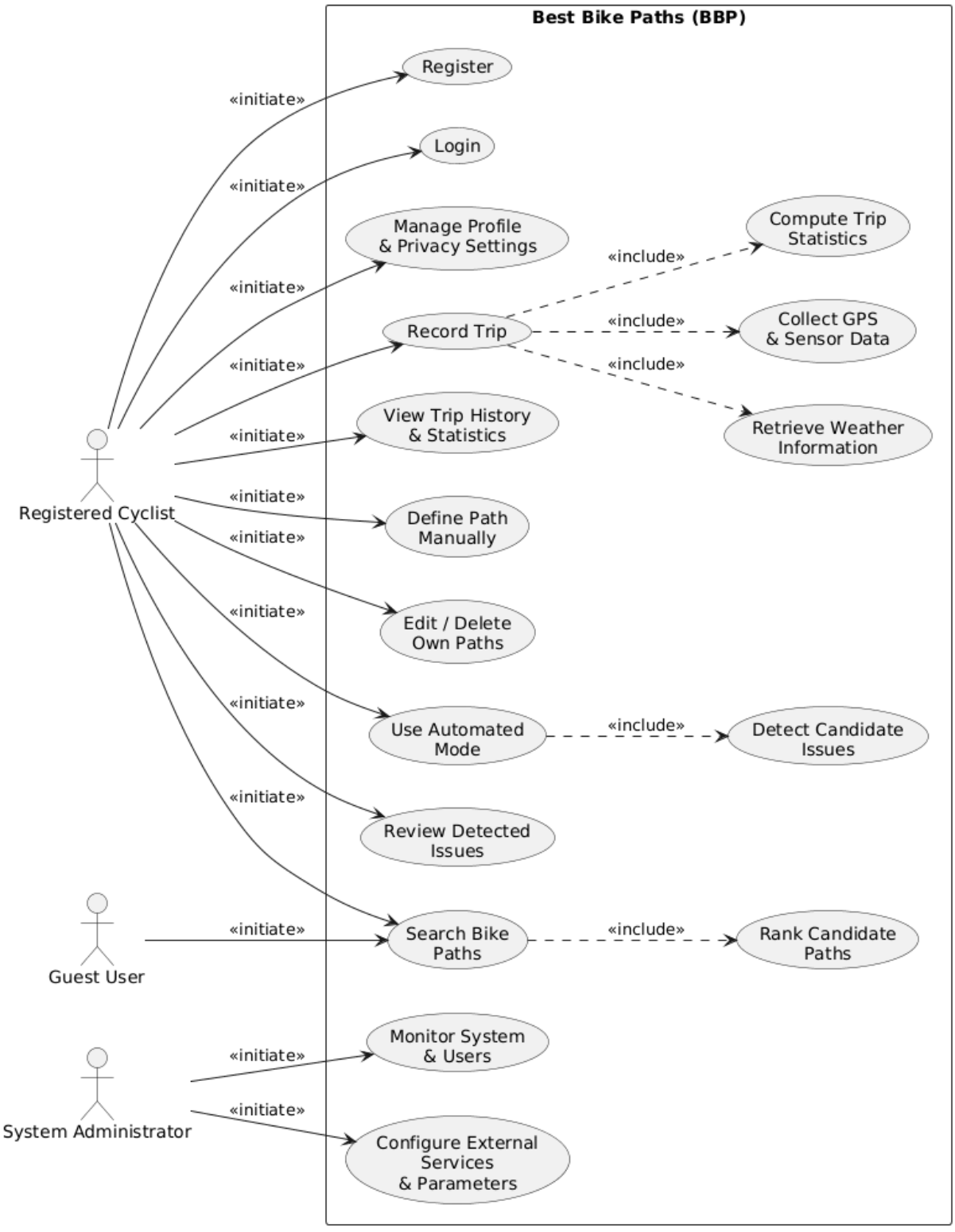



Figure 7: BBP Use Case Diagram

### 3.2.3 Use Cases

In this section, the behaviour of the system for the most relevant use cases is described in detail. Each use case is related to one or more functional requirements listed in Section 3.2.1.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

| | |
|---|---|
| **Name** | Register as a User |
| **Actors** | Registered Cyclist |
| **Entry Condition** | The cyclist is not logged in and does not already have an account. |
| **Exit Condition** | A new user account is created and can be used to log in to BBP. |
| **Event Flow** | 1. The cyclist accesses the registration page.<br>2. The system prompts the cyclist to enter the required information (e-mail and password).<br>3. The cyclist submits the registration form.<br>4. The system verifies the data and checks that the e-mail address is not already in use.<br>5. Upon successful verification, the system creates the user account and confirms registration. |
| **Exceptions** | • The e-mail address is already in use.<br>• Required fields are missing or not valid. |
| **Related Requirements** | R1, R4 |
| **Special Requirements** | The system must ensure secure data transmission during registration (e.g., HTTPS) and enforce basic password complexity rules. |

Table 2: Use Case: Register as a User

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

| | |
|---|---|
| **Name** | Log in as a User |
| **Actors** | Cyclist |
| **Entry Condition** | The cyclist has a previously created account and is not currently logged in. |
| **Exit Condition** | The cyclist is authenticated and redirected to the personal dashboard. |
| **Event Flow** | 1. The cyclist opens the login page.<br>2. The system prompts for e-mail and password.<br>3. The cyclist enters the credentials and submits the form.<br>4. The system validates the credentials.<br>5. Upon successful validation, the system redirects the cyclist to the dashboard. |
| **Exceptions** | • Invalid e-mail or password.<br>• The account has been disabled by an administrator. |
| **Related Requirements** | R2 |
| **Special Requirements** | The system must use encryption to secure user credentials during login and should mitigate brute-force attacks (e.g., rate limiting, temporary lock-out). |

Table 3: Use Case: Log in as a User

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

| | |
|---|---|
| **Name** | Record a Trip |
| **Actors** | Registered Cyclist |
| **Entry Condition** | The cyclist is authenticated and has granted the application access to location and motion sensors. |
| **Exit Condition** | A new trip with statistics and optional weather information is stored in the cyclist's trip history. |
| **Event Flow** | 1. The cyclist opens the trip recording view.<br>2. The cyclist presses "Start" to begin recording.<br>3. The system periodically acquires GPS samples (and sensor data in automated mode) and displays current speed, elapsed time, and distance.<br>4. If network connectivity is not available, the system buffers the collected data locally.<br>5. At the end of the ride, the cyclist presses "Stop".<br>6. The system computes trip statistics (distance, duration, average speed).<br>7. The system queries the external weather service, when available, and attaches the relevant information to the trip.<br>8. The system stores the trip and its statistics in the cyclist's history. |
| **Exceptions** | • Location services are disabled or permission is revoked during recording.<br>• GPS data are temporarily unavailable; the system interpolates or marks missing segments.<br>• The weather service is not reachable; the trip is stored without weather information. |
| **Related Requirements** | R6, R7, R8, R9, R11, R21 |
| **Special Requirements** | The system must minimise battery consumption during recording and must not lose already collected trip data in case of temporary network failures or application crashes. |

Table 4: Use Case: Record a Trip

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

| | |
|---|---|
| **Name** | Define a Path Manually |
| **Actors** | Registered Cyclist |
| **Entry Condition** | The cyclist is authenticated. |
| **Exit Condition** | A new path with status and optional obstacles is stored as private or publishable information. |
| **Event Flow** | 1. The cyclist opens the manual path definition view.<br>2. The system displays the map centred on a chosen area.<br>3. The cyclist selects the first segment to start the path and then repeatedly adds only segments that connect to the last selected one until the route is complete.<br>4. The cyclist assigns a status to the path or to individual segments.<br>5. The cyclist optionally adds obstacles along the path (type and approximate location).<br>6. The cyclist chooses whether the path shall be publishable or remain private.<br>7. The cyclist saves the path.<br>8. The system stores the path and its annotations in the inventory. |
| **Exceptions** | • The path contains no segments.<br>• Mandatory information (e.g., status) is missing.<br>• User selects a non-connecting segment. |
| **Related Requirements** | R12, R13, R14, R15, R16, R27, R28 |
| **Special Requirements** | The map interface must remain responsive while the user selects segments and must provide clear visual feedback for selected segments, obstacles, and publication status. |

Table 5: Use Case: Define a Path Manually

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

| | |
|---|---|
| **Name** | Review Detected Issues |
| **Actors** | Registered Cyclist |
| **Entry Condition** | The cyclist has recorded at least one trip in automated mode that produced candidate issues. |
| **Exit Condition** | Candidate issues are either validated (possibly corrected) or discarded, and the inventory is updated accordingly. |
| **Event Flow** | 1. The cyclist opens the detected issues review view.<br>2. The system lists candidate issues grouped by trip.<br>3. For each issue, the cyclist inspects its location and proposed type on the map.<br>4. The cyclist may confirm the issue, correct its attributes (e.g., type or position), or reject it.<br>5. When the cyclist finishes the review, the system stores confirmed issues as obstacles and removes rejected ones. |
| **Exceptions** | • There are no candidate issues to review.<br>• A network error occurs while saving the review results. |
| **Related Requirements** | R20, R22, R23, R24, R25, R26, R27, R28 |
| **Special Requirements** | The interface must clearly distinguish between confirmed, corrected and rejected issues and must not publish any automatically detected issue before explicit user confirmation. |

Table 6: Use Case: Review Detected Issues

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

| | |
|---|---|
| **Name** | Search Bike Paths Between Origin and Destination |
| **Actors** | Guest User, Registered Cyclist |
| **Entry Condition** | The user has access to the path search view. |
| **Exit Condition** | Either (a) one or more ranked candidate paths are displayed or (b) the system informs the user that no candidate path is available. |
| **Event Flow** | 1. The user specifies an origin and a destination on the map or via textual input.<br>2. The user submits the query.<br>3. The system validates the input and resolves addresses to coordinates if needed.<br>4. The system selects candidate bike paths connecting origin and destination using publishable information from the inventory.<br>5. For each candidate, the system computes a path score.<br>6. The system ranks candidate paths based on their score.<br>7. The system displays the ranked paths on the map along with summaries (distance, estimated time, score, number of obstacles). |
| **Exceptions** | • Origin or destination cannot be resolved.<br>• No candidate path can be found between origin and destination.<br>• The map service is temporarily unavailable. |
| **Related Requirements** | R29, R30, R31, R32, R33, R34, R35 |
| **Special Requirements** | Path search should complete within a few seconds under normal load, and only publishable information must be used to compute and display candidate paths. |

Table 7: Use Case: Search Bike Paths Between Origin and Destination

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

| | |
|---|---|
| **Name** | Manage Profile and Privacy Settings |
| **Actors** | Registered Cyclist |
| **Entry Condition** | The cyclist is authenticated. |
| **Exit Condition** | The cyclist's profile information and publication preferences are updated. |
| **Event Flow** | 1. The cyclist opens the settings/profile view.<br>2. The system retrieves the current profile information and privacy preferences.<br>3. The system displays editable fields and privacy options (default visibility for newly created paths and issues).<br>4. The cyclist edits profile data and/or privacy settings.<br>5. The cyclist confirms the changes.<br>6. The system validates the provided data.<br>7. The system updates the profile and privacy settings in persistent storage.<br>8. The system shows a confirmation message. |
| **Exceptions** | • Some mandatory fields are not valid.<br>• The old password for a password change does not match the stored one.<br>• A temporary error occurs while saving the updated profile. |
| **Related Requirements** | R4, R5 |
| **Special Requirements** | Changes in privacy settings affect only future contributions unless the cyclist explicitly edits past paths or issues. The updated profile must be consistently applied across all devices used by the same account. |

Table 8: Use Case: Manage Profile and Privacy Settings

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

| | |
|---|---|
| **Name** | View Trip History and Statistics |
| **Actors** | Registered Cyclist |
| **Entry Condition** | The cyclist is authenticated and has recorded at least one trip. |
| **Exit Condition** | The cyclist can see the list of past trips and the details of a selected trip. |
| **Event Flow** | 1. The cyclist opens the trip history view.<br>2. The system retrieves the list of trips associated with the cyclist.<br>3. The system displays the list with basic information (date, distance, duration).<br>4. The cyclist optionally filters or sorts the list.<br>5. The cyclist selects a trip.<br>6. The system shows detailed information for the selected trip, including a map of the path, statistics (distance, duration, average speed) and, if available, the weather information. |
| **Exceptions** | • The cyclist has no recorded trips; the system displays an appropriate message.<br>• A temporary error occurs while retrieving the trip list or details. |
| **Related Requirements** | R8, R9, R10, R11 |
| **Special Requirements** | The interface must remain responsive even when the cyclist has a large number of trips. Access to trip history shall be limited to the owner of the trips. |

Table 9: Use Case: View Trip History and Statistics

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

| | |
|---|---|
| **Name** | Use Automated Mode |
| **Actors** | Registered Cyclist |
| **Entry Condition** | The cyclist has installed the mobile application and granted the required permissions for accessing location and motion sensors. |
| **Exit Condition** | Automated mode is enabled or disabled, and the system is configured to infer biking activity and detect candidate issues. |
| **Event Flow** | 1. The cyclist opens the settings or recording view where automated mode can be controlled.<br>2. The system shows the current status of automated mode (enabled/disabled).<br>3. The cyclist toggles automated mode to the desired state.<br>4. If enabling automated mode, the system checks that required permissions (location, motion sensors, background activity) are granted.<br>5. The system activates or deactivates background processes that monitor speed and sensor data to infer biking activity.<br>6. When automated mode is active and biking is detected, the system starts collecting candidate trip and issue data.<br>7. When biking activity is no longer detected for a sufficient time, the system stops collecting data for the candidate trip. |
| **Exceptions** | • Required permissions are not granted; the system prompts the cyclist to enable them.<br>• The operating system prevents background activity (e.g., power-saving mode).<br>• Device resources are insufficient to keep automated mode running. |
| **Related Requirements** | R17, R18, R19, R20, R21 |
| **Special Requirements** | Automated mode shall minimise false positives and false negatives while keeping battery consumption within acceptable limits. Users must always be able to disable automated mode. |

Table 10: Use Case: Use Automated Mode

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

| | |
|---|---|
| **Name** | Edit or Delete Own Paths |
| **Actors** | Registered Cyclist |
| **Entry Condition** | The cyclist is authenticated and has at least one manually defined path. |
| **Exit Condition** | The selected path is updated or removed from the inventory according to the cyclist's request. |
| **Event Flow** | 1. The cyclist opens the view listing their own paths.<br>2. The system retrieves and displays the list of owned paths.<br>3. The cyclist selects a path.<br>4. The cyclist chooses either to edit or to delete the path.<br>5. If editing:<br>(a) The system displays the path on the map with its status and obstacles.<br>(b) The cyclist modifies segments, status, obstacles or publication flag.<br>(c) The cyclist saves the changes.<br>(d) The system validates the updated information and stores the modified path.<br>6. If deleting:<br>(a) The system asks for confirmation.<br>(b) Upon confirmation, the system deletes the path and its annotations from the inventory. |
| **Exceptions** | • The selected path cannot be found (e.g., it was deleted from another device).<br>• Mandatory information is missing or invalid after editing.<br>• A temporary error occurs while saving or deleting the path. |
| **Related Requirements** | R15, R16, R27, R28 |
| **Special Requirements** | Updates to a publishable path shall be applied atomically so that other users never see inconsistent intermediate versions. Deleting a path shall ensure that it is no longer used for future path search queries. |

Table 11: Use Case: Edit or Delete Own Paths

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

| | |
|---|---|
| **Name** | Monitor System and Users |
| **Actors** | System Administrator |
| **Entry Condition** | The administrator is authenticated in the administrative web interface. |
| **Exit Condition** | The administrator has inspected system status and, if necessary, taken actions such as disabling problematic accounts. |
| **Event Flow** | 1. The administrator accesses the monitoring dashboard.<br>2. The system retrieves aggregated statistics about users, trips, paths, and obstacles.<br>3. The system displays key indicators (number of active users, recent trips, error rates, etc.).<br>4. The administrator may filter data or inspect detailed logs.<br>5. The administrator selects a specific user account, if needed.<br>6. The system displays the user profile and recent activity.<br>7. The administrator may decide to disable or re-enable an account, or reset certain attributes.<br>8. The system applies the requested changes and updates the monitoring views. |
| **Exceptions** | • Monitoring data or logs are temporarily unavailable.<br>• A requested administrative action fails due to database or communication errors. |
| **Related Requirements** | R1, R2, R4, R5, R10, R27, R28 |
| **Special Requirements** | Only administrators shall have access to this functionality. Administrative actions must be logged for audit purposes, including timestamp and administrator identity. |

Table 12: Use Case: Monitor System and Users

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

| | |
|---|---|
| **Name** | Configure External Services and Parameters |
| **Actors** | System Administrator |
| **Entry Condition** | The administrator is authenticated in the administrative web interface. |
| **Exit Condition** | Configuration parameters and external service credentials are updated and active. |
| **Event Flow** | 1. The administrator opens the configuration view.<br>2. The system retrieves the current configuration parameters (e.g., map provider keys, weather service keys, thresholds for automated detection).<br>3. The system displays editable configuration fields grouped by category.<br>4. The administrator modifies one or more parameters or external service credentials.<br>5. The administrator saves the configuration changes.<br>6. The system validates the values (e.g., checks format, required fields).<br>7. For external services, the system may perform a connectivity check using the new credentials.<br>8. The system applies the new configuration and stores it in persistent storage. |
| **Exceptions** | • A parameter value is invalid (e.g., out of allowed range).<br>• Connectivity tests to an external service fail using the new credentials.<br>• A temporary error occurs while saving configuration changes. |
| **Related Requirements** | R27, R29, R30, R31, R32, R33 |
| **Special Requirements** | Configuration changes affecting core functionality should require explicit confirmation and may trigger a short service interruption. All changes must be logged, and the system should support rolling back to a previous configuration if necessary. |

Table 13: Use Case: Configure External Services and Parameters

### 3.2.4 Sequence Diagrams

For the main use cases described in Section 3.2.3, a sequence diagram specifies the interaction between the user, the client application, and the backend services. The following figures report the sequence diagrams for the most relevant use cases of BBP.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

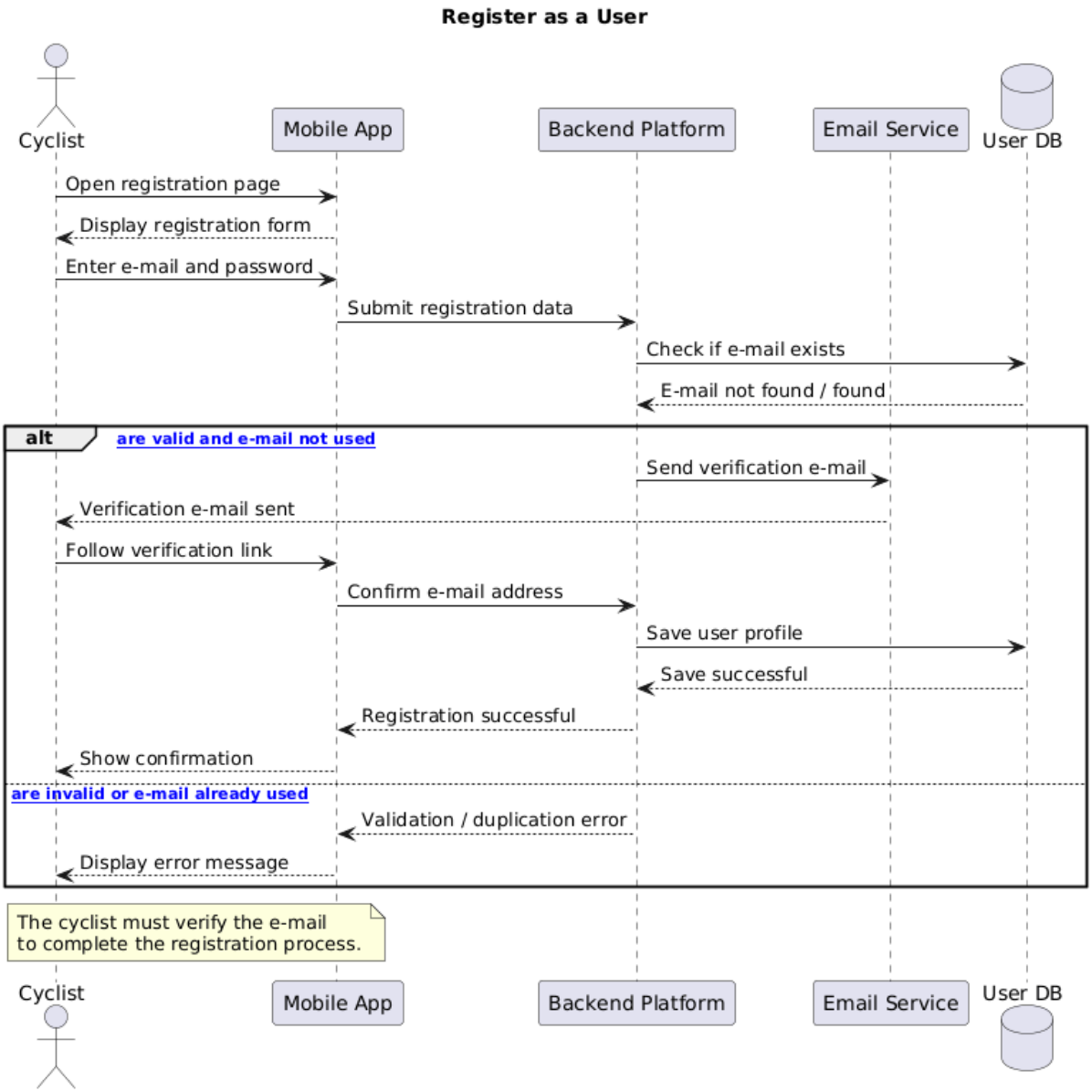



Figure 8: Sequence diagram for “Register as a User”

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

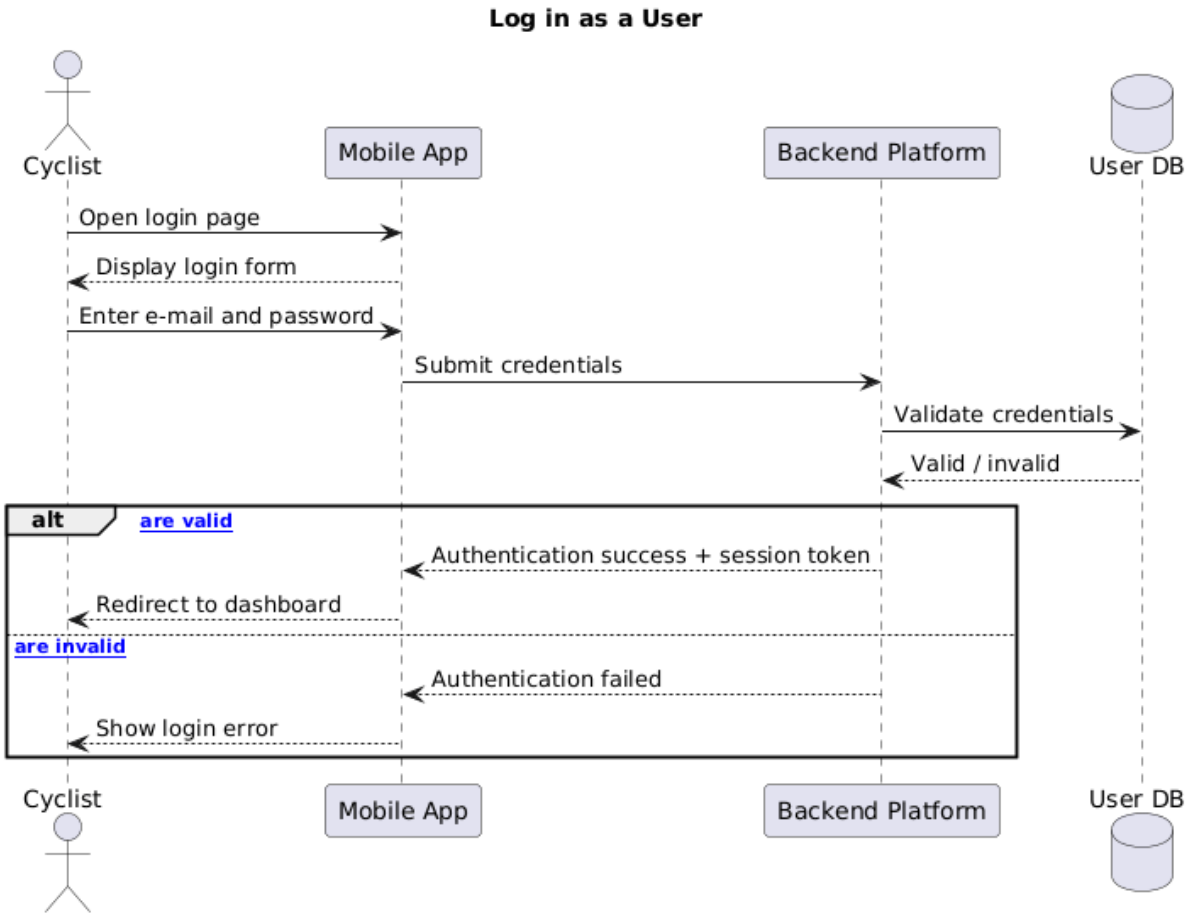



Figure 9: Sequence diagram for “Log in as a User”

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

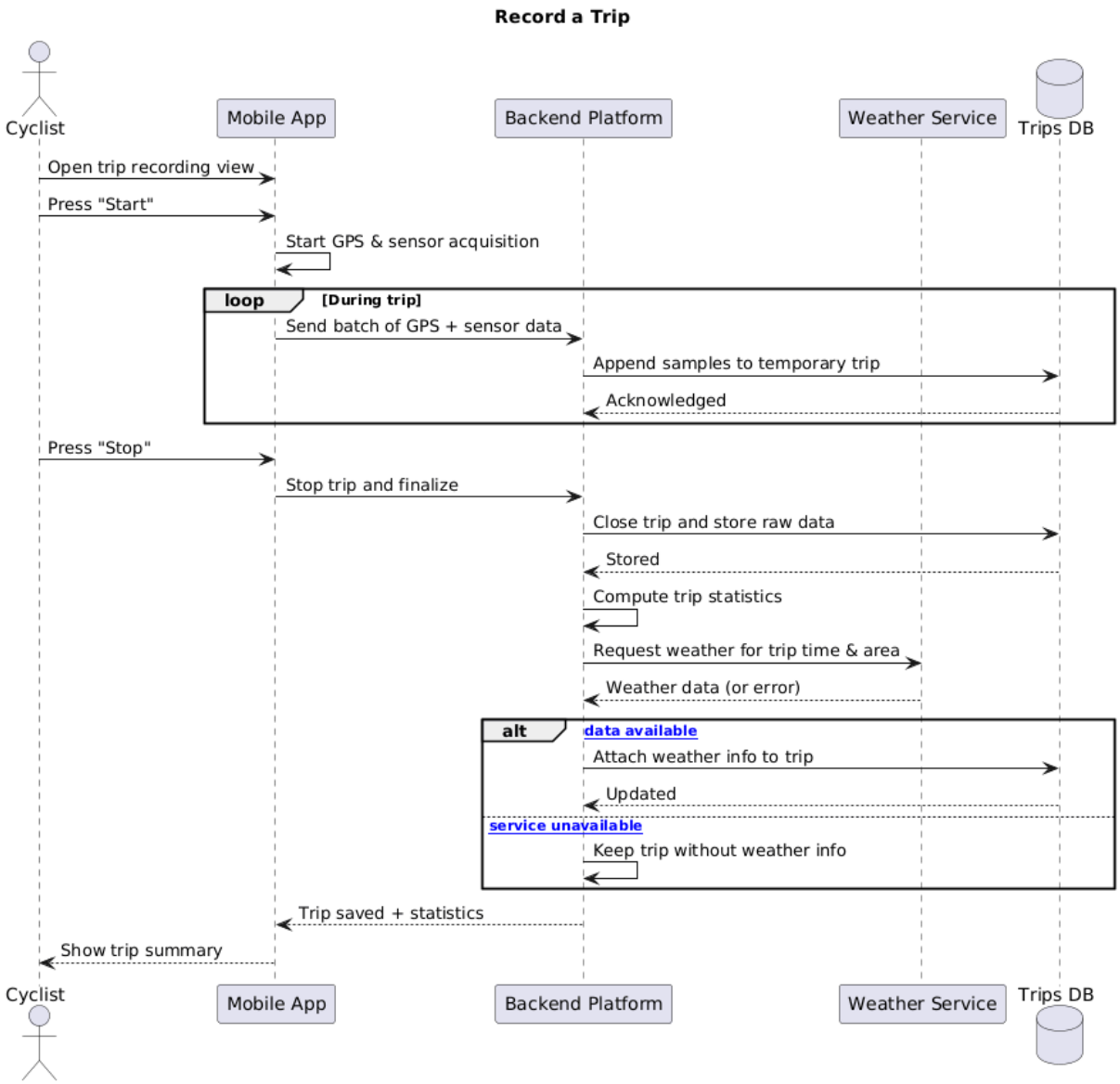



Figure 10: Sequence diagram for “Record a Trip”

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

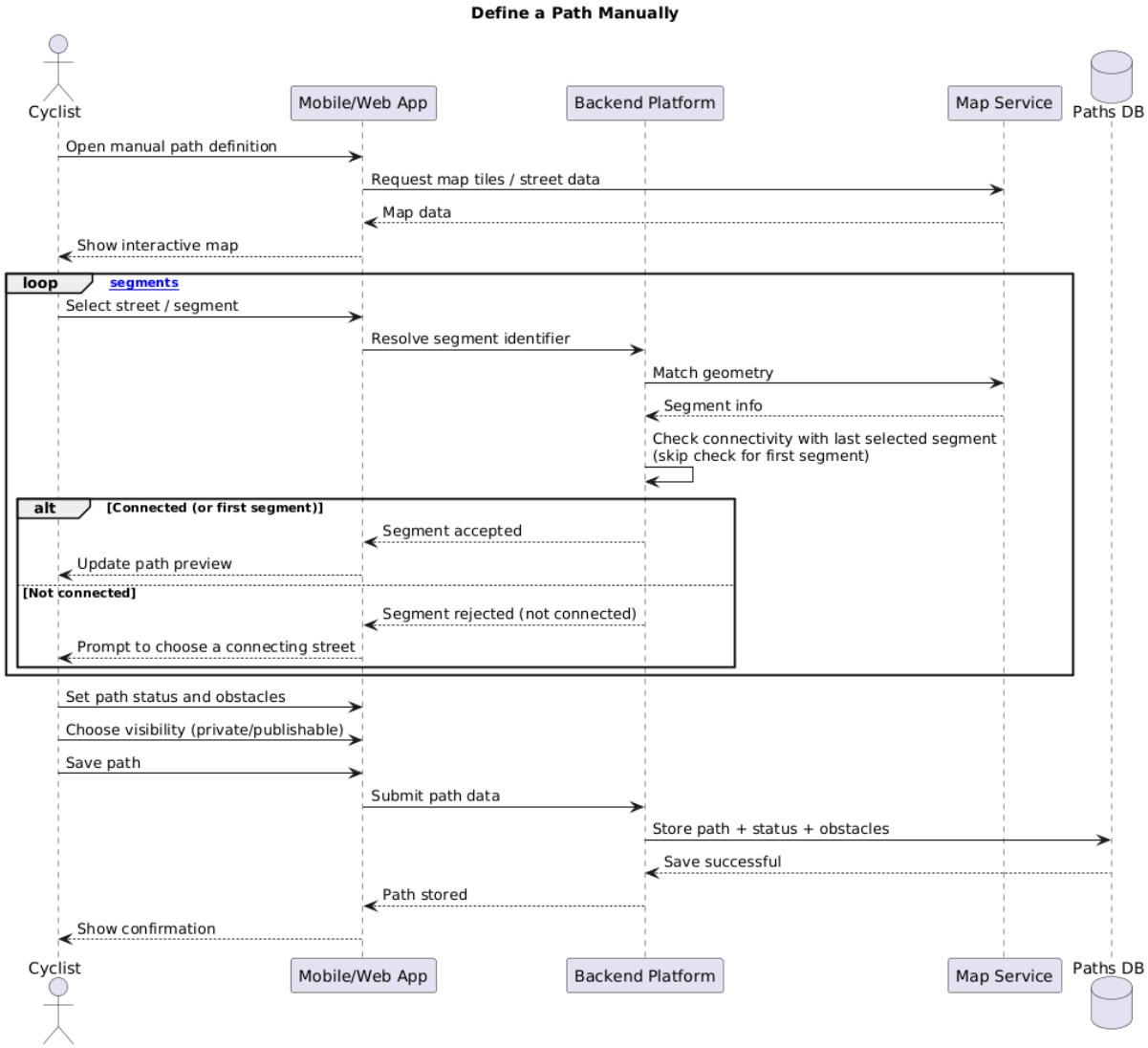



Figure 11: Sequence diagram for “Define a Path Manually”

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

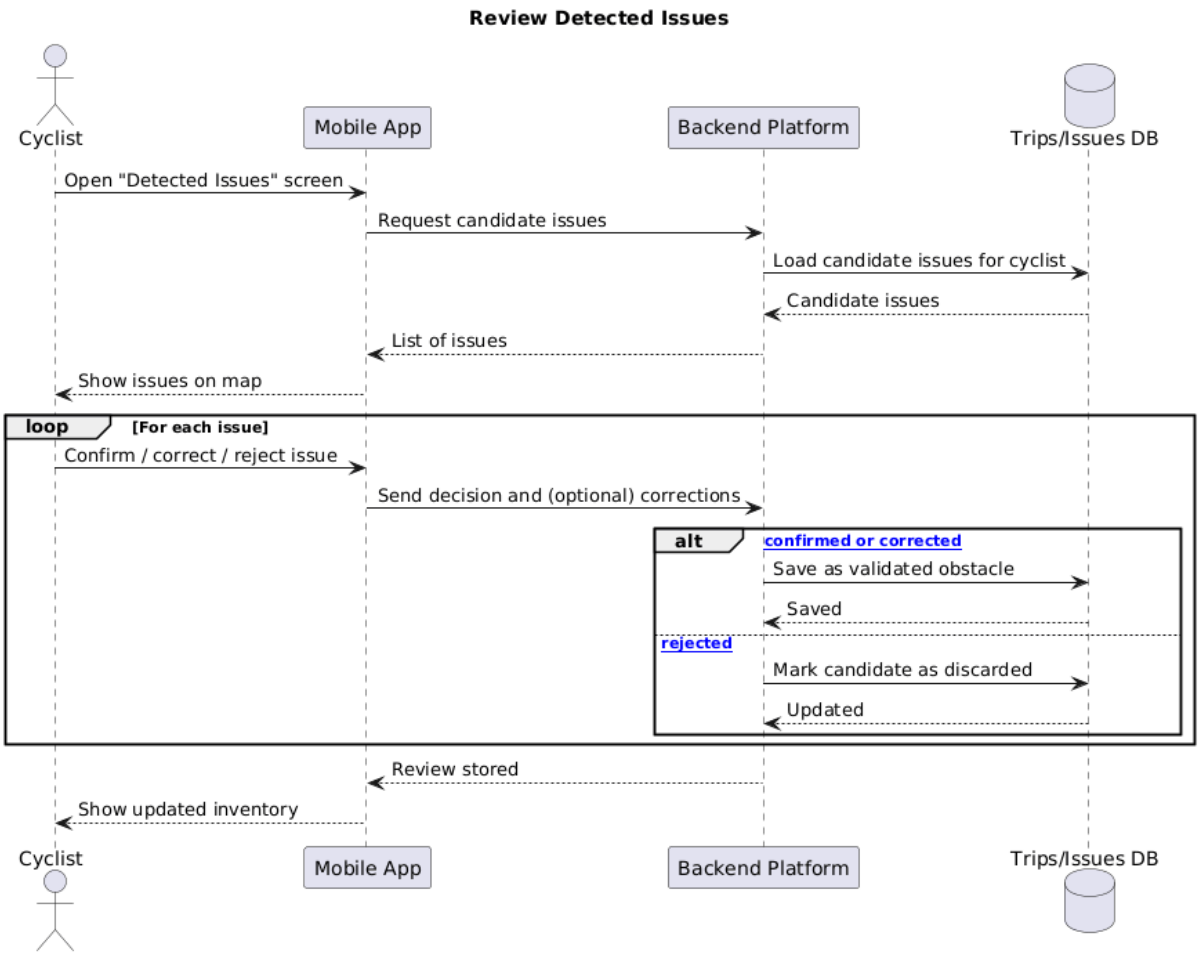



Figure 12: Sequence diagram for “Review Detected Issues”

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

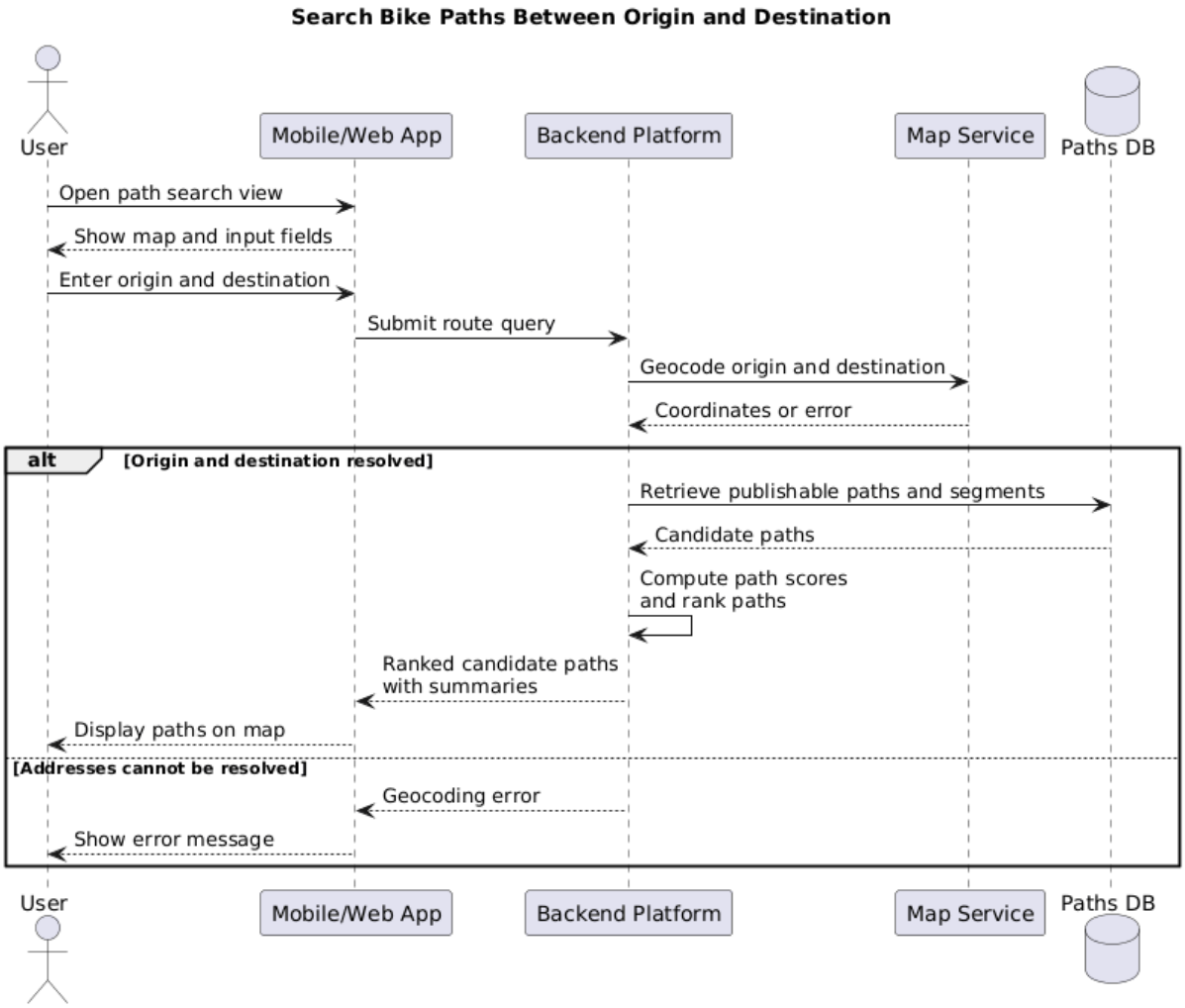



Figure 13: Sequence diagram for “Search Bike Paths Between Origin and Destination”

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

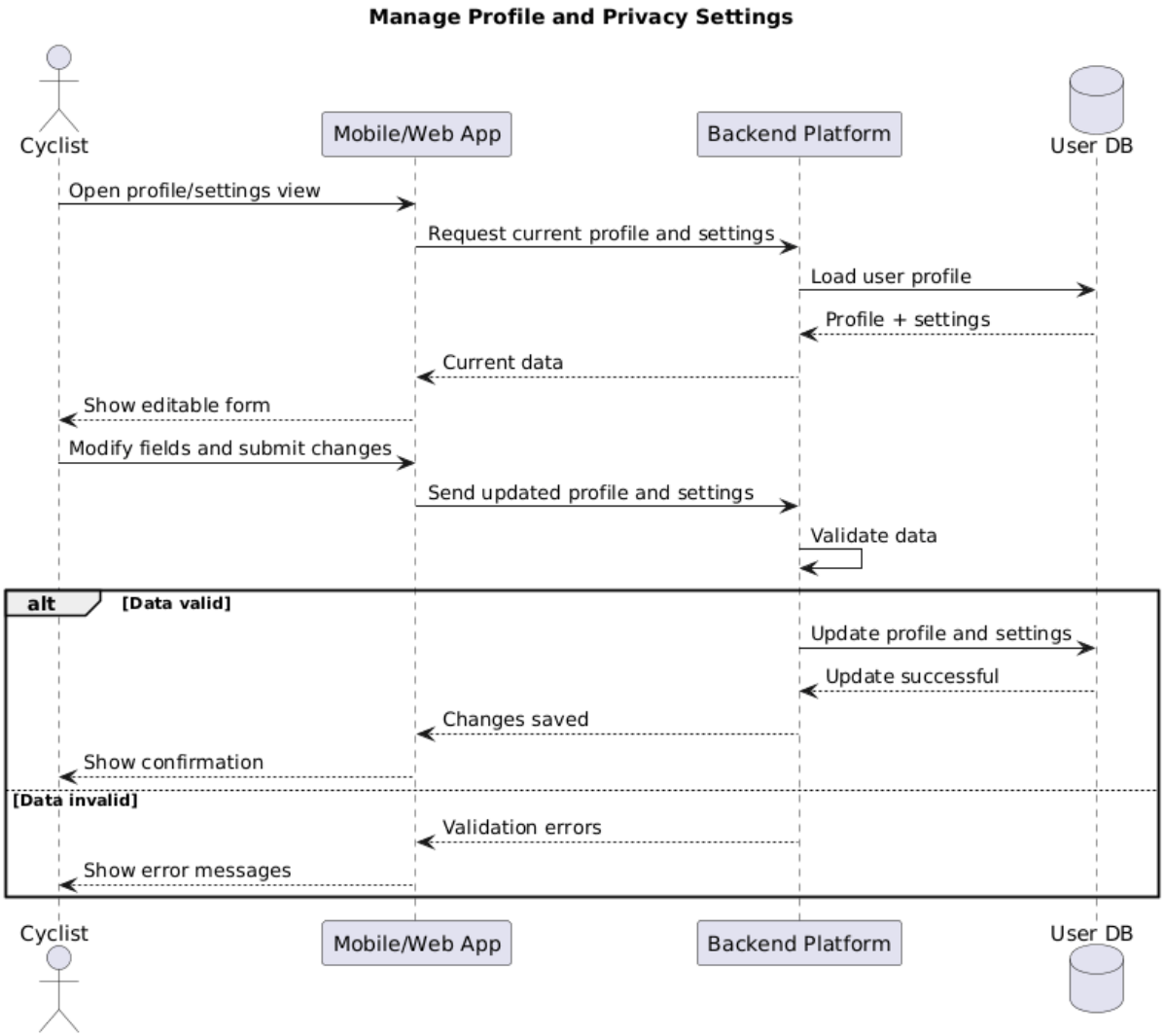



Figure 14: Sequence diagram for “Manage Profile and Privacy Settings”

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

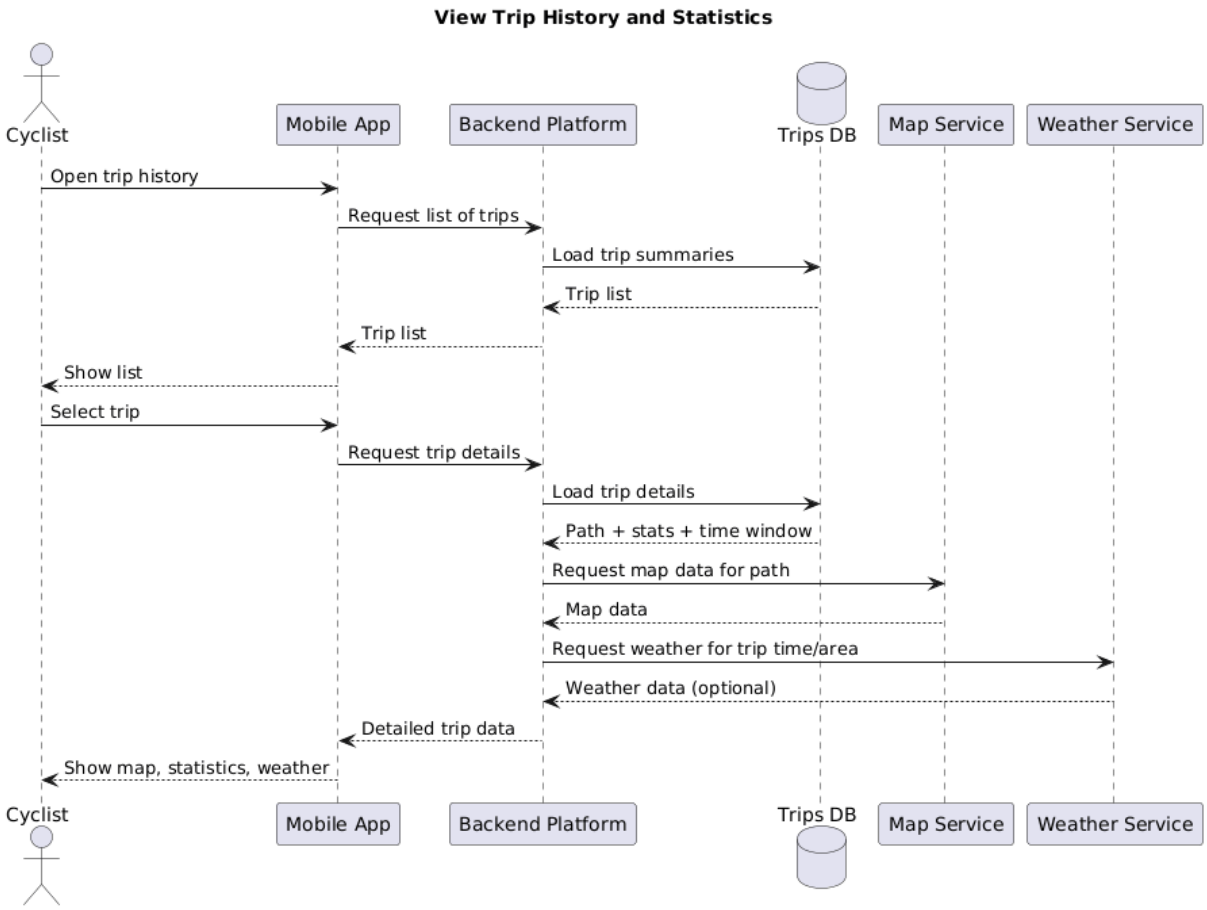



Figure 15: Sequence diagram for “View Trip History and Statistics”

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

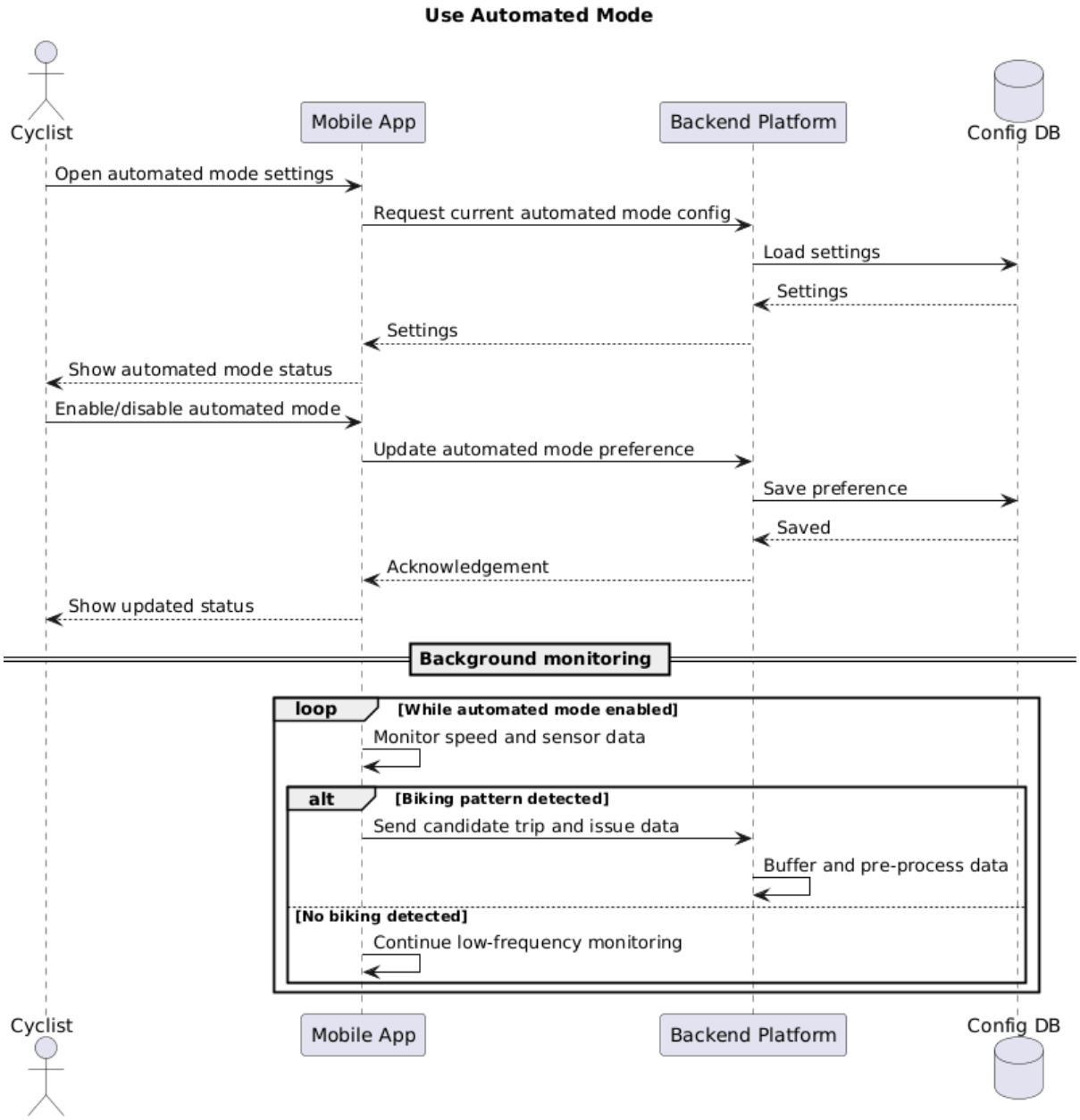



Figure 16: Sequence diagram for “Use Automated Mode”

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

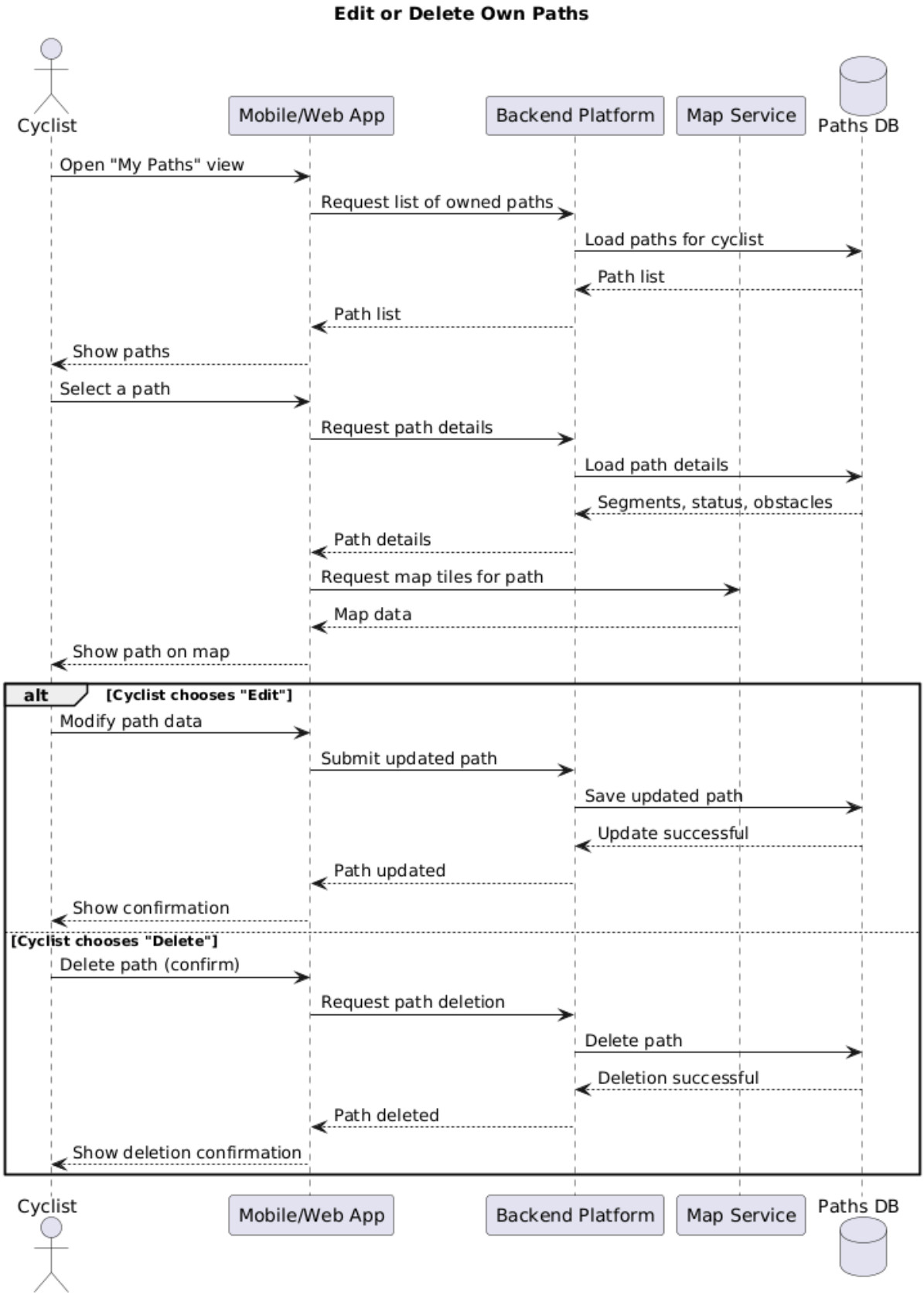



Figure 17: Sequence diagram for “Edit or Delete Own Paths”

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

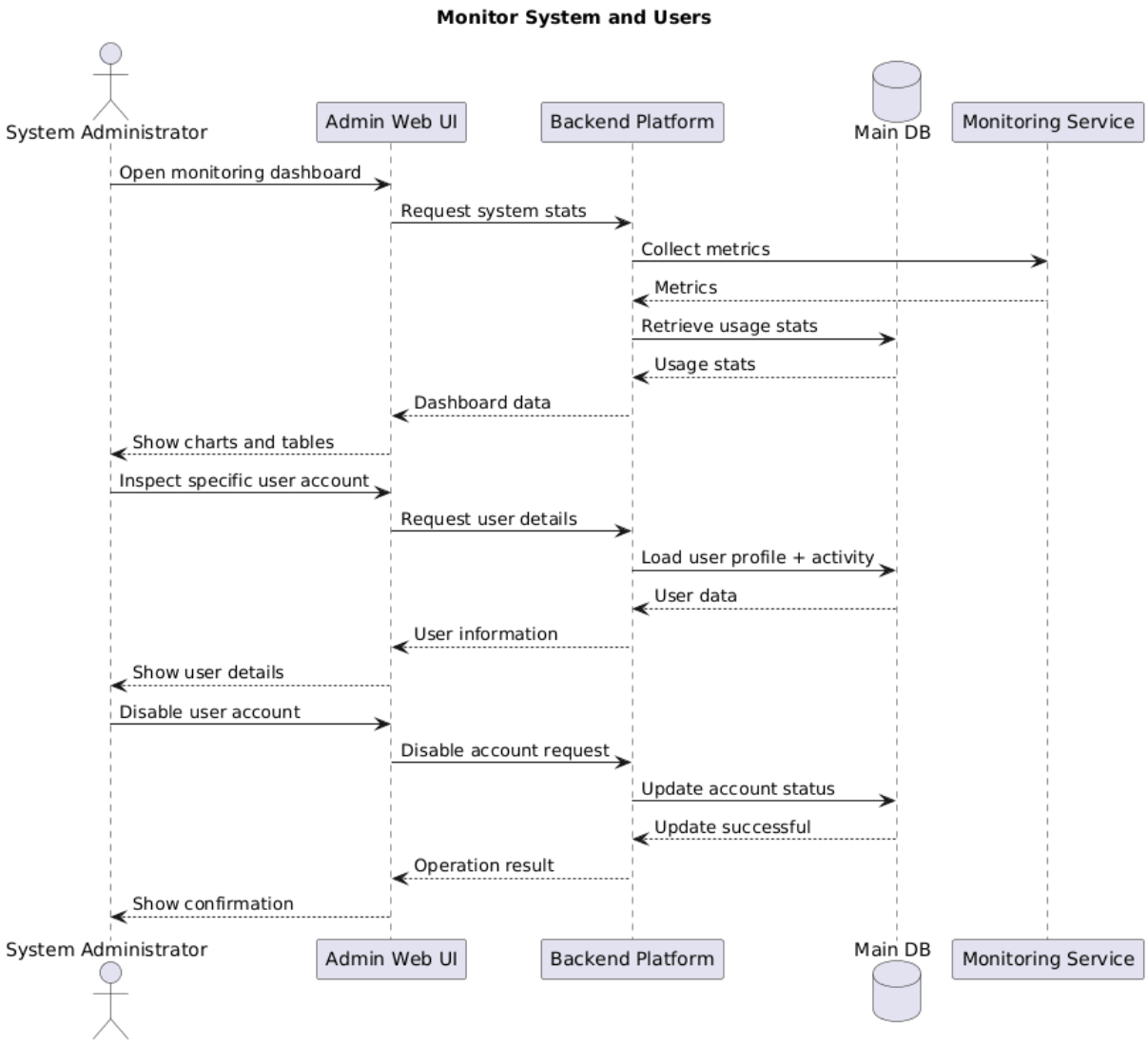



Figure 18: Sequence diagram for “Monitor System and Users”

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

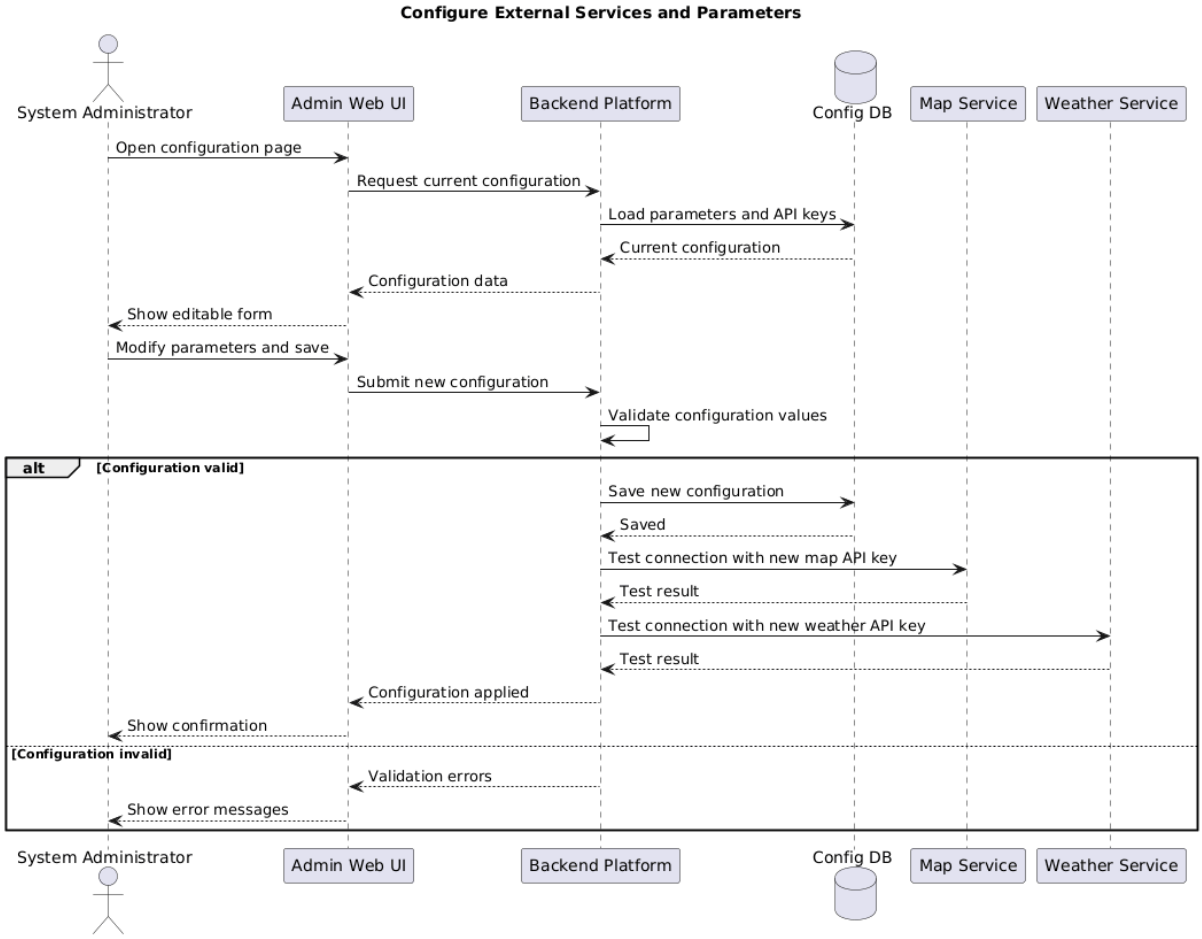



Figure 19: Sequence diagram for “Configure External Services and Parameters”

### 3.2.5 Requirements Mapping

Table 14 summarizes how the functional requirements support the high-level goals defined in Section **??**.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

| Goal (G) | Requirements (R) | Assumptions (D) |
|---|---|---|
| G1: Trip Recording | R6, R7, R8, R9, R21 | D1, D2 |
| G2: Trip Statistics and History | R8, R9, R10, R11 | D1, D3 |
| G3: Manual Path Definition | R12, R13, R14, R15, R16, R27, R28 | D3, D4 |
| G4: Automated Path Data Acquisition | R17, R18, R19, R20, R21 | D1, D5, D6 |
| G5: Confirmation of Inferred Information | R22, R23, R24, R25, R26 | D5, D6 |
| G6: Path Inventory and Publication | R15, R16, R26, R27, R28, R29 | D3, D4, D7 |
| G7: Path Search and Visualization | R30, R31, R34, R35 | D3, D4, D7 |
| G8: Path Scoring and Recommendation | R32, R33, R35 | D3, D4, D7, D8 |

Table 14: Mapping of Goals to Requirements and Domain Assumptions

### 3.3 Performance Requirements

The following requirements describe the expected performance of the BBP system.

- **[PR1] Path search response time.**
  For at least 90% of the path search queries, the system shall return a ranked list of candidate paths within **5 seconds**; the worst case response time shall not exceed **10 seconds** under normal operating conditions (average load).

- **[PR2] Trip recording latency.**
  During trip recording, the mobile application shall acquire and store GPS samples with an effective frequency of at least **1 sample per second**. The user interface shall update the displayed distance and time at least once per second without noticeable delays.

- **[PR3] Data upload and synchronization.**
  After a trip is stopped, the system shall upload the recorded data to the backend and store the trip in the cyclist's history within **30 seconds** for trips shorter than **2 hours**, assuming a stable network connection.

- **[PR4] Scalability and concurrent users.**
  The backend shall support at least **100 concurrent active users** performing typical operations (recording trips, searching paths, reviewing issues) without violating PR1–PR3.

- **[PR5] Availability of core services.**
  The services responsible for path search and trip storage shall be operational at least **99% of the time** over a one-month period, excluding scheduled maintenance windows.

- **[PR6] Storage capacity.**
  The system shall be able to store at least **5 years** of historical trip data and path information for the expected user base without requiring manual purging of data.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

## 3.4 Design Constraints

### 3.4.1 Standards compliance

- The communication between client applications and the backend shall use the **HTTP/HTTPS** protocols. All requests carrying authentication credentials or personal data *must* use **HTTPS** with up-to-date versions of the **TLS** protocol.
- Data exchanged through the public API shall use widely adopted formats such as **JSON** and **UTF-8** encoding.
- The system shall comply with applicable **data protection and privacy regulations** (e.g., GDPR-like principles), meaning that users must be informed about the data collected and must be able to request deletion of their personal data.
- The mobile application shall follow the basic **usability guidelines** of the target platforms (Android, iOS) regarding navigation patterns, permissions handling, and notification usage.

### 3.4.2 Hardware limitations

- The mobile application shall run on typical mid-range smartphones available on the market at the time of development, with at most **4 GB of RAM** and limited CPU resources.
- The application shall be designed to minimize **battery consumption** while recording trips, for example by adapting the sampling rate and avoiding unnecessary background computation.
- The system shall tolerate temporary **loss of GPS signal** and **network connectivity**, buffering data locally and synchronizing it when possible.
- No dedicated hardware or sensors other than the standard smartphone components (GPS, accelerometer, gyroscope, network connectivity) shall be required from the user.

### 3.4.3 Other constraints

- The backend shall be implemented as a set of **modular services**, to ease future evolution and deployment on different infrastructures (e.g., cloud platforms).
- The persistent storage shall be implemented using a widely used **relational or document-oriented DBMS**; the choice of a specific product is left to the design and implementation phase.
- The system will rely on one or more **third-party map** and **weather** providers; therefore, the design shall isolate interactions with these services behind well-defined interfaces to ease future replacement.
- The user interface texts and messages shall be prepared to support **internationalization** (e.g., English as default language with the possibility of adding other languages).

## 3.5 Software System Attributes

### 3.5.1 Reliability

- The system shall ensure that recorded trips and manually defined paths are either stored completely and correctly or not stored at all (no partially inconsistent records).
- In case of temporary failures of external services (e.g., weather or map provider), the system shall still allow core operations such as recording trips and searching paths, possibly without enriched information.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

- The backend shall support periodic **data backup** procedures so that in case of catastrophic failure it is possible to restore the system with at most **24 hours** of data loss.

### 3.5.2 Availability

- Core services (authentication, trip storage, path search) shall be designed to support an overall availability of at least **99%** over a one-month period, excluding scheduled maintenance windows announced in advance.
- The system shall degrade gracefully under high load (e.g., by limiting the rate of requests per user) rather than becoming completely unavailable.

### 3.5.3 Security

- Access to non-public data (trip histories, manually defined private paths, account information) shall require **authentication**.
- The system shall enforce **authorization** rules so that users can modify or delete only the information they own; administrators may have additional privileges.
- All sensitive data in transit shall be protected using **TLS** (see D.1), and sensitive data at rest (such as password hashes) shall be stored using **industry-standard hashing and salting** mechanisms.
- The system shall implement basic protections against common web vulnerabilities (e.g., SQL injection, cross-site scripting) by using validated frameworks and input validation.

### 3.5.4 Maintainability

- The system shall follow a modular architecture that separates concerns (e.g., presentation layer, business logic, data access, external services) to facilitate changes.
- Source code shall follow consistent **coding conventions** and be documented at least at module and public interface level.
- The system shall include **logging** and **basic monitoring** facilities, so that abnormal situations can be diagnosed without access to users' personal devices.
- Automated tests (unit and integration tests) shall cover the most critical components (e.g., path search, trip storage, user management) to support regression testing.

### 3.5.5 Portability

- The mobile client shall support at least one major mobile platform (e.g., Android) and be designed in such a way that porting it to another platform (e.g., iOS) does not require a complete rewrite of the business logic.
- The web client shall be compatible with recent versions of the main desktop browsers supporting HTML5, CSS3, and modern JavaScript.
- The backend shall not depend on platform-specific features of the hosting environment so that it can be deployed on different infrastructures (e.g., on-premise server or cloud provider) with minimal changes.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

# 4 Formal Analysis Using Alloy

This section describes the formal analysis conducted using Alloy to model and verify some of the core functionalities of the BBP platform. It focuses on the functionalities related to recording trips by cyclists, automatically detecting potential issues on the infrastructure, confirming those issues and storing them as obstacles in the shared path inventory, and finally answering route queries with candidate bike paths and a recommended path. Using Alloy 6 and its temporal operators, we validate that issues follow the intended lifecycle (NewIssue → Pending → Confirmed/Rejected), that published obstacles always come from confirmed issues, and that recommended routes are consistent with the scoring function and the visibility constraints of the paths.

Here is the Alloy code:

```
//-----------
// Signatures
//-----------

open util/integer
open util/temporal

// Registered cyclist of the BBP platform
sig Cyclist {
    var trips  : set Trip,
    var paths  : set Path,
    var issues : set DetectedIssue
}

// Basic piece of infrastructure
sig PathSegment {}

// Status assigned to a path
enum PathStatus { Safe, Medium, Unsafe }

// Visibility of a path in the shared inventory
enum Visibility { Private, Publishable }

// Path contributed by a cyclist and stored in the inventory
sig Path {
    owner      : one Cyclist,
    segments   : some PathSegment,
    var status : one PathStatus,
    var visibility : one Visibility
}

// Recorded trip made by a cyclist
sig Trip {
    cyclist  : one Cyclist,
    segments : some PathSegment
}
```

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

```
// Lifecycle of automatically detected issues
enum IssueState { NewIssue, Pending, Confirmed, Rejected }

// Potential obstacle detected on a segment during a trip
sig DetectedIssue {
    reporter : one Cyclist,
    trip     : one Trip,
    segment  : one PathSegment,
    var state: one IssueState
}

// Obstacle stored in the shared path inventory
sig Obstacle {
    source  : one DetectedIssue,
    segment : one PathSegment
}

// Abstract locations for route queries
sig Location {}

// Route query between origin and destination
sig RouteQuery {
    origin, destination : one Location,
    var candidates      : set CandidatePath,
    var recommended     : lone CandidatePath
}

// Candidate path returned for a query
sig CandidatePath {
    path  : one Path,
    query : one RouteQuery,
    score : one Int
}



//---------------------------------
// Facts: basic relationships
//---------------------------------

fact OwnershipFacts {

    // Each path is owned by exactly one cyclist and appears in her paths
    always all p: Path | one c: Cyclist | {
        p.owner = c
        p in c.paths
    }

    // Each trip belongs to exactly one cyclist and appears in her trips
    always all t: Trip | one c: Cyclist | {
```

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

```
        t.cyclist = c
        t in c.trips
    }

    // Each issue belongs to its reporter and refers to her trips
    always all i: DetectedIssue | {
        i in i.reporter.issues
        i.trip in i.reporter.trips
        i.segment in i.trip.segments
    }

    // Each obstacle keeps the same segment as its source issue
    always all o: Obstacle | {
        o.segment = o.source.segment
    }

    // Each candidate path is attached to exactly one query
    always all cp: CandidatePath | one q: RouteQuery | {
        cp.query = q
        cp in q.candidates
    }

    // If a query has a recommended candidate, it must be one of its candidates
    always all q: RouteQuery |
        some q.recommended implies q.recommended in q.candidates
}



//---------------------------------
// Facts: issues and obstacles
//---------------------------------

fact IssueAndObstacleFacts {

    // An issue can have at most one obstacle derived from it
    always all i: DetectedIssue |
        lone o: Obstacle | o.source = i

    // Only confirmed issues can give rise to obstacles
    always all i: DetectedIssue |
        (some o: Obstacle | o.source = i) implies i.state = Confirmed

    // For non-confirmed issues there is no obstacle yet
    always all i: DetectedIssue |
        i.state in (NewIssue + Pending + Rejected) implies
            no o: Obstacle | o.source = i
}
```

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

```
//---------------------------------
// Facts: lifecycle of issue state
//---------------------------------

fact IssueLifecycleRules {

    // From NewIssue we may stay New, go Pending, Confirmed, or Rejected
    always all i: DetectedIssue |
        i.state = NewIssue implies
            after i.state in (NewIssue + Pending + Confirmed + Rejected)

    // From Pending we may stay Pending, or become Confirmed or Rejected
    always all i: DetectedIssue |
        i.state = Pending implies
            after i.state in (Pending + Confirmed + Rejected)

    // Once Confirmed, the issue remains Confirmed
    always all i: DetectedIssue |
        i.state = Confirmed implies
            after i.state = Confirmed

    // Once Rejected, the issue remains Rejected
    always all i: DetectedIssue |
        i.state = Rejected implies
            after i.state = Rejected
}



//---------------------------------
// Facts: paths and publishability
//---------------------------------

fact PathVisibilityAndStatus {

    // A path that contains an obstacle cannot be "Safe"
    always all p: Path |
        (some o: Obstacle | o.segment in p.segments) implies
            p.status != Safe

    // Publishable paths must not include segments with unconfirmed issues
    always all p: Path |
        p.visibility = Publishable implies
            no i: DetectedIssue |
                i.segment in p.segments and i.state in (NewIssue + Pending)

    // Paths referenced by candidate routes must be publishable
    always all cp: CandidatePath |
        cp.path.visibility = Publishable
```

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

```
}



//--------------------------
// Facts: recommendations
//--------------------------

fact RecommendationRules {

    // If a query has a recommended candidate, it must have maximal score
    always all q: RouteQuery |
        some q.recommended implies {
            let r = q.recommended |
                all cp: q.candidates | r.score >= cp.score
        }

    // Scores of candidates of the same query are not all equal
    always all q: RouteQuery |
        some q.candidates implies
            some disj c1, c2: q.candidates | c1.score != c2.score
}



//--------------------
// Setup and Execution
//--------------------

pred show {
    // Small scope for visualising trips, issues and obstacles
    #Cyclist       = 3
    #Path          = 3
    #Trip          = 2
    #PathSegment   = 5
    #DetectedIssue = 4
    #Obstacle      = 0
    #RouteQuery    = 0
    #CandidatePath = 0

    // Force a visible trace: start with new issues, then evolve
    all i: DetectedIssue | i.state = NewIssue
    eventually some i: DetectedIssue | i.state = Pending
    eventually some i: DetectedIssue | i.state = Confirmed
    eventually some i: DetectedIssue | i.state = Rejected
}

run show for 6 but 10 steps
```

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

```
pred show2 {
    // Scope for a single route query with candidates
    #Cyclist       = 2
    #Path          = 3
    #Trip          = 0
    #PathSegment   = 5
    #DetectedIssue = 0
    #Obstacle      = 0
    #RouteQuery    = 1
    #CandidatePath = 3

    // Make sure we actually see a recommendation
    one q: RouteQuery | one q.recommended
}

run show2 for 6 but 5 steps
```

## 4.1 Results

First, we visualize that detected issues in the BBP model follow the intended lifecycle progression.

The trace is generated by executing the `show` predicate under a bounded temporal simulation (`10 steps`) with a small scope of cyclists, trips, paths, segments, and issues. In the initial step, all reported issues start in the *NewIssue* state, representing newly reported observations on trip segments. In the subsequent steps, the model exhibits the expected evolution of issue states: at least one issue eventually moves to *Pending*, and later an issue reaches the *Confirmed* state, while another ends in *Rejected*. This confirms that the temporal constraints encoded in `IssueLifecycleRules` are satisfiable together with the structural facts connecting cyclists, trips, segments, and issues. Moreover, the trace remains consistent with the safety and publishability constraints on paths, ensuring that the instance is not only satisfiable but also aligned with the intended semantics of the BBP platform.

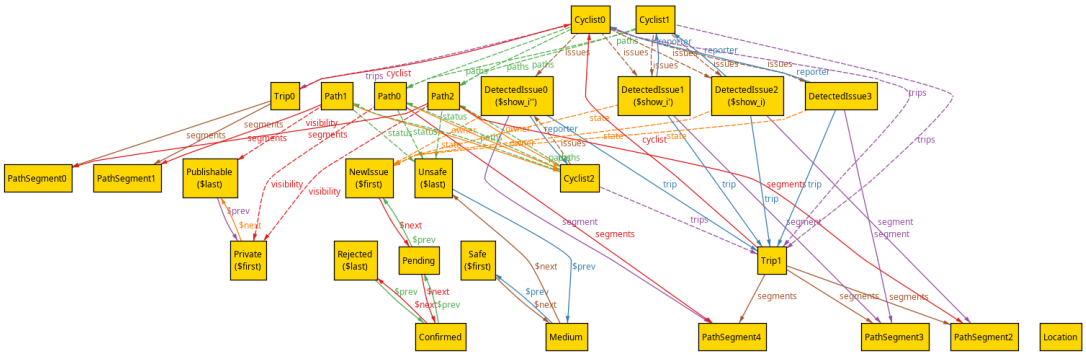



Figure 20: Temporal trace for `show`: evolution of detected issues across steps (from *NewIssue* to *Pending*, *Confirmed*, or *Rejected*).

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

Next, we analyze the recommendation scenario using the `show2` predicate. Here, we construct a single `RouteQuery` with multiple `CandidatePath` options, each associated with a score, and we enforce the existence of exactly one recommended candidate. The resulting instance demonstrates that the recommended path is selected from among the candidates, and, as required by the recommendation rules, it corresponds to a maximal score relative to the other candidate paths. This execution therefore illustrates the correctness of the recommendation-related constraints in isolation, and confirms that the model can produce a valid recommendation even in a minimal query setting.

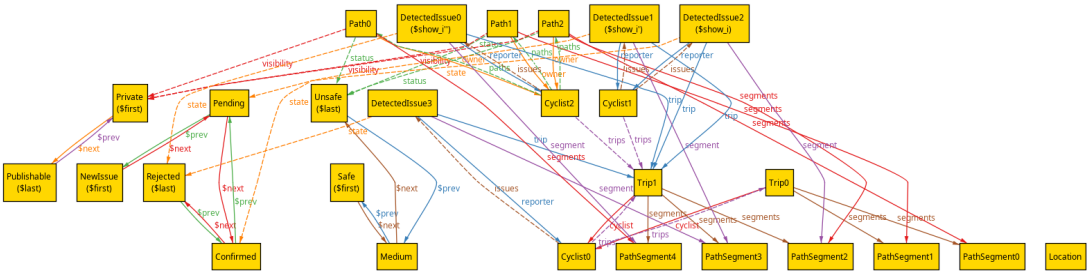



Figure 21: Recommendation scenario for `show2`: one route query with multiple candidate paths and a single recommended candidate satisfying the maximal-score constraint.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

# 5 Effort Spent

This section includes information about how much effort each group member spent in working on this document.

- *Hamidreza Khodadadi*

| Chapter | Hours |
|---|---|
| 1 | 8 |
| 2 | 5 |
| 3 | 14 |
| 4 | 10 |

- *Mohammad Rashid Khan Mofrad*

| Chapter | Hours |
|---|---|
| 1 | 8 |
| 2 | 19 |
| 3 | 9 |
| 4 | 7 |

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 

# 6 References

- Overleaf as LATEX editor
- GitHub for project versioning
- PlantUML for UML diagrams
- Alloy for formal analysis

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved 